

Cloud Computing

CAT7002 Course Notes

Servers, virtualization, data centres and the addressing that connects them.

by Bhuvnesh Verma

Contents

Unit 1: Virtualization Concepts and Hypervisors	2
Unit 1: Virtualization Concepts and Hypervisors	2
1.1 Introduction to Servers	2
1.1.1 Types of servers	2
1.2 The Traditional Server Model	2
1.2.1 Advantages	3
1.2.2 Disadvantages	3
1.3 Introduction to Virtualization	4
1.3.1 Architecture diagram	4
1.3.2 Reasons for using virtualization	5
1.4 Hypervisor	6
1.4.1 Architecture diagram	6
1.4.2 Type 1 hypervisor	6
1.4.3 Type 2 hypervisor	6
1.4.4 Type 1 versus Type 2	7
1.5 Full Virtualization	8
1.5.1 Architecture diagram	8
1.6 Paravirtualization	8
1.6.1 Architecture diagram	8
1.7 Hardware Based Virtualization	9
1.7.1 Architecture diagram	9
1.8 Types of Virtualization	10
1.8.1 Overview	10
1.8.2 Desktop virtualization	10
1.8.3 Network virtualization	10
1.8.4 Server and machine virtualization	11
1.8.5 Storage virtualization	11
1.8.6 Operating system level virtualization	11
1.8.7 Application virtualization	11
1.9 Levels of Virtualization	12
1.9.1 Before and after virtualization	12
1.9.2 Implementation levels	12
1.9.3 Instruction set architecture level	13
1.9.4 Hardware abstraction level	13
1.9.5 Operating system level	13
1.9.6 Library support level	13
1.9.7 User-application level	13

1.10 Containers	14
1.10.1 Containers versus virtual machines	14
1.10.2 Comparison	15
1.11 Container Orchestration	15
1.11.1 Why it is needed	15
Unit 2: Virtualization Technologies and Data Center Management	16
Unit 2: Virtualization Technologies and Data Center Management	16
2.1 Virtualization Technologies	16
2.2 Software Virtualization	17
2.2.1 Components	17
2.2.2 Types of software virtualization	17
2.3 Operating System Virtualization	18
2.3.1 Architecture diagram	18
2.3.2 Types of virtual disk	18
2.3.3 Advantages	19
2.3.4 Disadvantages	19
2.4 Application Virtualization	20
2.4.1 Architecture diagram	20
2.4.2 Benefits	20
2.5 Service Virtualization	21
2.5.1 Architecture diagram	21
2.6 Hardware Virtualization	22
2.6.1 Benefits	22
2.7 Storage Virtualization	23
2.7.1 Types of storage virtualization	23
2.7.2 Benefits	23
2.8 The Data Center	24
2.8.1 Need for a data centre	24
2.8.2 Objectives of a data centre	24
2.8.3 Components	24
2.9 High Availability of a Data Center	25
2.9.1 Architecture diagram	25
2.9.2 Need for high availability	25
2.9.3 Causes of data centre downtime	25
2.10 Data Center Maintenance	26
2.10.1 Why it matters	26

2.11	Planned Maintenance	27
2.11.1	Objectives	27
2.11.2	Characteristics	27
2.11.3	Activities	28
2.11.4	Advantages	28
2.11.5	Disadvantages	28
2.12	Unplanned Maintenance	29
2.12.1	Causes	29
2.12.2	Common activities	29
2.12.3	Steps followed	29
2.12.4	Challenges	30
2.12.5	Best practices to reduce it	30
2.12.6	Planned versus unplanned maintenance	30
2.13	Server Migration	31
2.13.1	Benefits	31
2.13.2	Types of server migration	32
2.13.3	Migration considerations	33
2.13.3.1	Business considerations	33
2.13.3.2	Hardware considerations	33
2.13.3.3	Software compatibility	33
2.13.3.4	Data integrity	33
2.14	Planning a Server Migration	34
2.14.1	Phase 1: Discovery	34
2.14.2	Phase 2: Assessment	34
2.14.2.1	Migration strategies	35
2.14.3	Phase 3: Migration	36
2.14.4	Best practices	36
2.14.5	Advantages of proper planning	36
2.15	Networking Concepts: Masks and CIDR	37
2.16	Subnetting	38
2.16.1	Levels of hierarchy	38
2.16.2	Finding the subnetwork address	38
2.17	Supernetting	40
2.18	Classless Addressing	41
2.18.1	Rules for allocating a block	41
2.18.2	Slash notation	41
2.18.3	Extracting information from an address	41
2.18.4	Address mask	41
2.19	Practice Problems	44
Unit 3:	Cloud Computing Fundamentals	52
Unit 3:	Cloud Computing Fundamentals	52

3.1	Importance of Cloud Computing	52
3.2	Cloud Service Models	53
3.2.1	Overview	53
3.3	Infrastructure as a Service (IaaS)	53
3.3.1	Architecture diagram	53
3.3.2	Benefits	54
3.3.3	Issues	54
3.3.4	Characteristics	54
3.4	Platform as a Service (PaaS)	54
3.4.1	Architecture diagram	55
3.4.2	Benefits	55
3.4.3	Issues	55
3.4.4	Characteristics	55
3.5	Software as a Service (SaaS)	55
3.5.1	Characteristics	56
3.5.2	Benefits	56
3.5.3	Issues	56
3.6	Cloud Deployment Models	57
3.6.1	Overview diagram	57
3.6.2	Public Cloud	57
3.6.3	Private Cloud	57
3.6.4	Hybrid Cloud	58
3.6.5	Community Cloud	58
3.7	Regions, Availability Zones and Edge Locations	59
3.7.1	Region	59
3.7.2	Availability Zone	60
3.7.3	Edge Location	60
3.7.4	Relationship between Region, AZ and Edge Location	61
3.7.5	Difference between Region, AZ and Edge Location	61
3.8	Cloud Computing: Benefits and Challenges	62
3.8.1	Benefits, in detail	62
3.8.2	The Challenges	64

Unit 1: Virtualization Concepts and Hypervisors

1.1. Introduction to Servers

Short description: A server is a computer or software system that provides services, resources, or data to other computers, called clients, over a network.

Everyday examples:

- Opening a website: the browser sends a request to a **web server**.
- Checking Gmail: a **mail server** delivers the messages.
- Using online banking: a **database server** stores and retrieves the account.

What a server does: Stores files, hosts websites, manages databases, runs applications, sends and receives email, shares printers and other resources, and manages users and security.

1.1.1 Types of servers

Server type	Purpose
Web server	Hosts websites
File server	Stores and shares files
Database server	Stores databases
Mail server	Sends and receives emails
Application server	Runs business applications
Print server	Manages printers
DNS server	Converts domain names into IP addresses

1.2. The Traditional Server Model

Short description: Before virtualization and cloud computing, organizations typically deployed **one physical server for one application**. Each application had its own dedicated hardware.

Application	Physical server
Website	Server 1
Database	Server 2
Email	Server 3
ERP	Server 4

1.2.1 Advantages

- **High performance.** Applications have direct access to hardware resources.
- **Better security.** Each application runs on its own physical machine.
- **Reliability.** No interference from other applications sharing the hardware.
- **Simple architecture.** Easy to install, configure and troubleshoot.
- **Full hardware control.** Administrators control the server hardware completely.

1.2.2 Disadvantages

- **Low resource utilization.** Most servers use only 10–20% of CPU and memory, leaving the rest idle.
- **High cost.** Each application requires separate hardware.
- **High power consumption.** More servers draw more electricity and need more cooling.
- **More physical space.** Multiple servers occupy valuable rack space.
- **Difficult scalability.** A new application often means buying new hardware.
- **Higher maintenance.** Every server needs updates, monitoring, backups and repairs.
- **Single point of failure.** If a physical server fails, its application becomes unavailable.

Why a new technology was needed

As organizations expanded they faced too many physical servers, rising hardware costs, increased electricity and cooling expenses, underutilized resources, complex management, and slow deployment of new services. These limitations led to **virtualization**, which allows multiple virtual servers to run on a single physical machine, and virtualization in turn became the foundation for **cloud computing**.

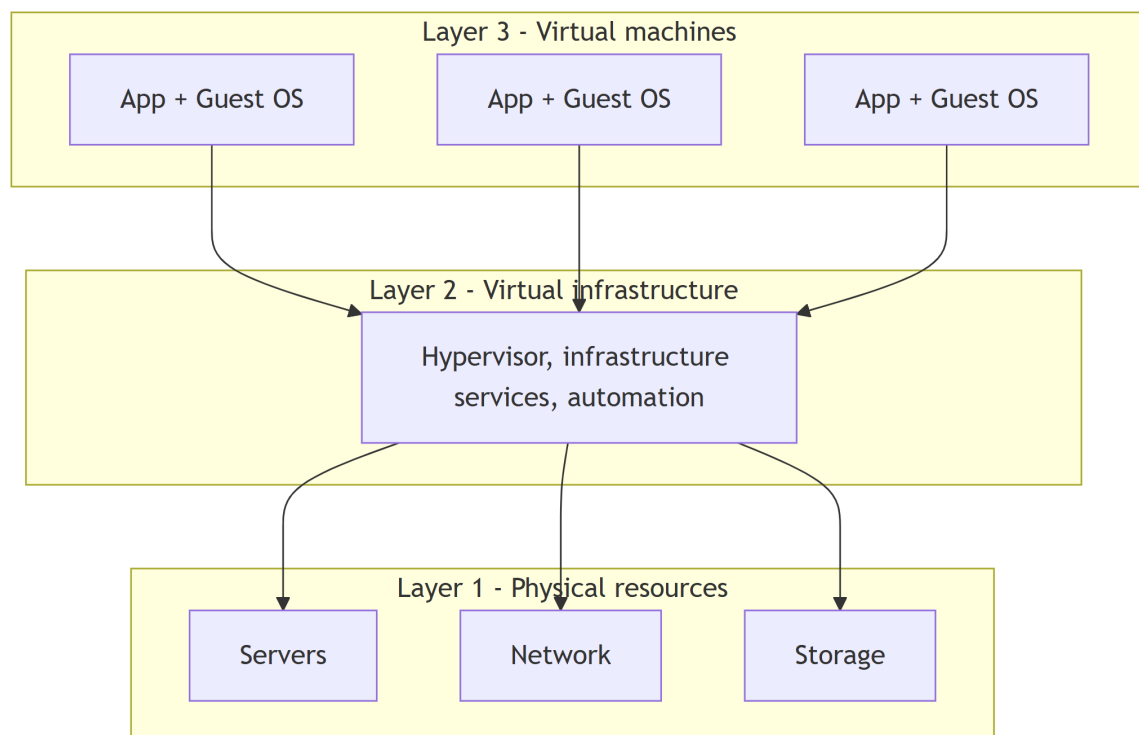
1.3. Introduction to Virtualization

Short description: Virtualization is a methodology for dividing computer resources into more than one execution environment, applying concepts such as partitioning, time-sharing, machine simulation and emulation.

Theory: Virtualization is a method in which multiple independent operating systems run on one physical computer. It maximizes the usage of available physical resources, so one can achieve high server usage: it increases total computing power while decreasing overhead. Increasing the number of logical operating systems on a single host reduces hardware acquisition and maintenance cost for an organization.

How it works: The architecture has three layers. Layer 1 is the physical infrastructure of servers, networks and storage. Layer 2 is the virtual infrastructure, consisting of hypervisors, virtual infrastructure services and automated solutions that optimize the IT process. Layer 3 holds the virtual machines, where different operating systems and applications are deployed. A single virtual infrastructure supports more than one virtual machine, so the physical resources of the whole infrastructure are shared across many virtual machines for maximum efficiency.

1.3.1 Architecture diagram



Notes

- Virtualization is a trend in IT that includes autonomic computing and utility computing: the environment manages itself, and every resource is seen as a utility that a client pays per use.
- It reduces the burden of workloads on users by centralizing administrative tasks and improving scalability.

1.3.2 Reasons for using virtualization

- Virtual machines consolidate the workloads of under-utilized servers, saving on hardware, environmental costs and management.
- Legacy applications can keep running inside a VM.
- A VM provides a secure sandbox for running an untrusted application.
- A VM helps in building a secured computing platform.
- A VM provides an illusion of hardware.
- A VM simulates networks of independent computers.
- A VM supports running distinct operating systems with different versions.

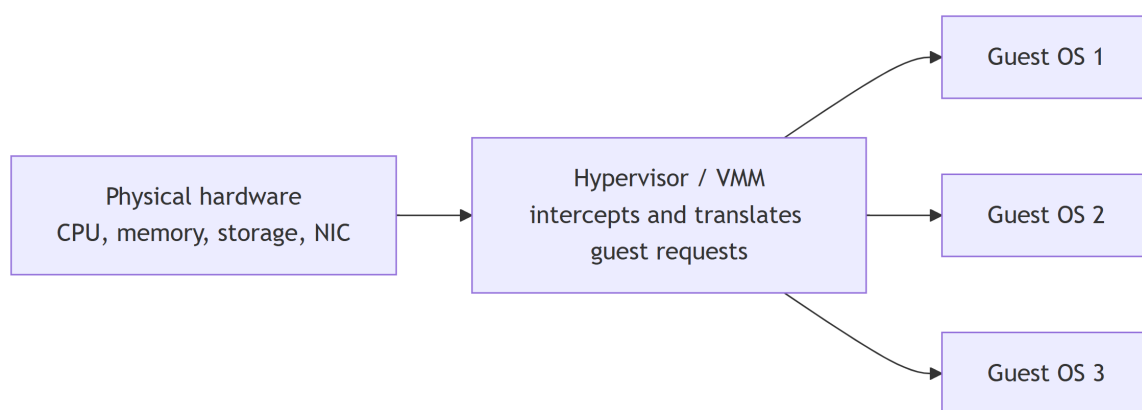
1.4. Hypervisor

Short description: A hypervisor, or Virtual Machine Monitor (VMM), is software that lets multiple operating systems run on a single physical machine.

Theory: It manages hardware resources such as CPU, memory and storage and allocates them to virtual machines without interference. This improves hardware utilization, reduces costs, and provides flexibility in cloud and server environments.

How it works: A hypervisor runs on hardware or on a host OS to create and manage virtual machines, each with its own virtual CPU, memory, storage and network. It intercepts guest OS requests and translates them to physical hardware, ensuring isolation, security and stability.

1.4.1 Architecture diagram



1.4.2 Type 1 hypervisor

A Type 1 hypervisor runs directly on the host's hardware and does not rely on a host operating system. This architecture offers better performance and security because there is no intermediary OS. It is the standard for enterprise-level data centres and for cloud providers such as Amazon Web Services and Microsoft Azure.

Examples: VMware ESXi, Microsoft Hyper-V, KVM (Kernel-based Virtual Machine), Xen.

- **Pros:** high performance through direct hardware access; strong security with no intermediate OS layer; suitable for mission-critical workloads.
- **Cons:** requires dedicated hardware; setup and management are complex compared to Type 2.

1.4.3 Type 2 hypervisor

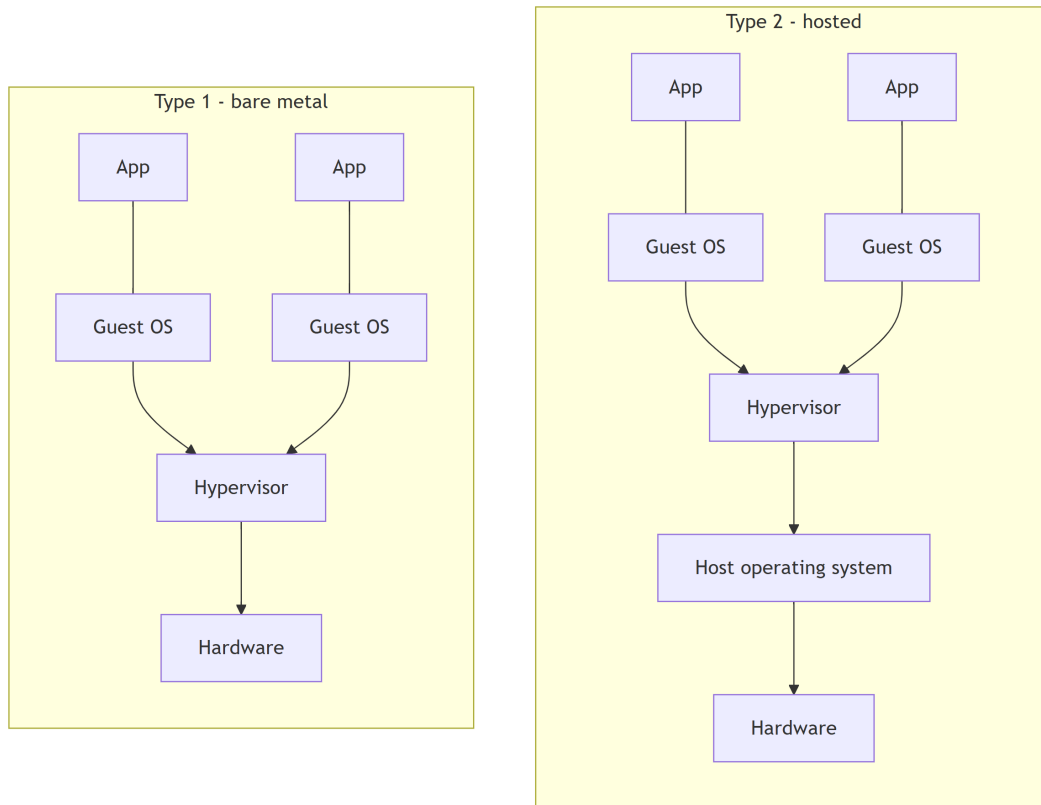
A Type 2 hypervisor runs on top of a conventional operating system such as Windows, macOS or Linux. It is essentially an application within the host OS. This type is generally used for desktop virtualization, development and testing, where a user needs to run multiple operating systems on a personal computer. Performance is slightly lower than Type 1 because of the overhead of the host OS.

Examples: Oracle VM VirtualBox, VMware Workstation, Parallels Desktop.

- **Pros:** easy to install and use; useful for development, testing and malware analysis; good host-guest integration features.

- **Cons:** slower performance with no direct hardware access; security depends on the host OS, so a compromise of the host may affect the guests.

1.4.4 Type 1 versus Type 2



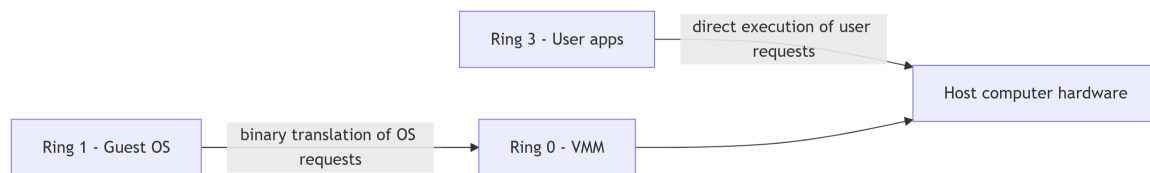
1.5. Full Virtualization

Short description: Full virtualization was introduced by IBM in 1966. It was the first software solution for server virtualization and uses binary translation together with direct execution.

How it works: The virtual machine completely isolates the guest OS from the virtualization layer and the hardware. Kernel code is translated to replace non-virtualizable instructions with new sequences of instructions that have the intended effect on the virtual hardware, while user level code is executed directly on the processor for high performance.

Examples: Microsoft and Parallels systems.

1.5.1 Architecture diagram



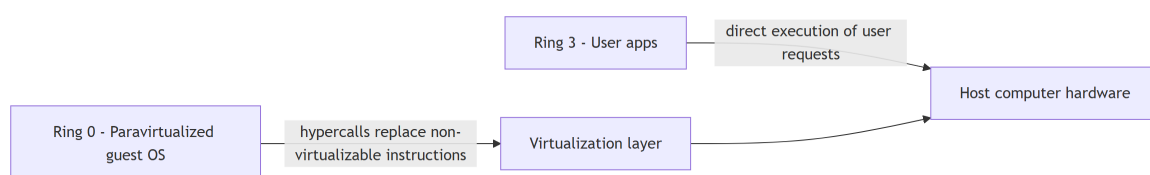
1.6. Paravirtualization

Short description: Paravirtualization is the category of CPU virtualization that uses hypercalls for operations handled at compile time.

How it works: The guest OS is not completely isolated, only partially isolated by the virtual machine from the virtualization layer and hardware. The OS kernel is modified to replace non-virtualizable instructions with hypercalls that communicate directly with the virtualization layer, which improves performance and efficiency.

Examples: VMware and Xen.

1.6.1 Architecture diagram

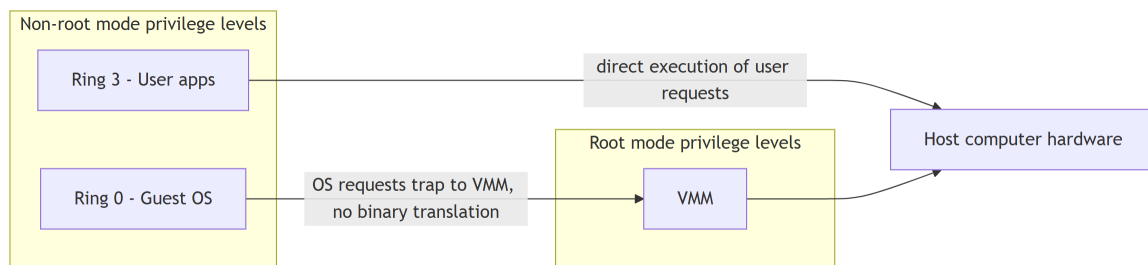


1.7. Hardware Based Virtualization

Short description: Hardware vendors added CPU features that simplify virtualization directly in silicon.

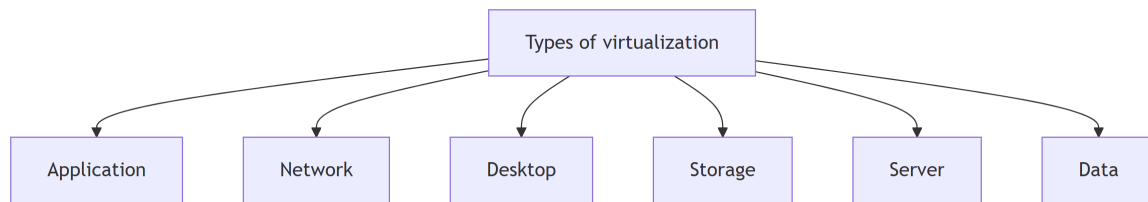
How it works: First generation enhancements include Intel Virtualization Technology (VT-x) and AMD's AMD-V. Both target privileged instructions with a new CPU execution mode that allows the VMM to run in a root mode below ring 0. Privileged and sensitive calls are set to trap automatically to the hypervisor, removing the need for either binary translation or paravirtualization. The guest state is stored in Virtual Machine Control Structures (VT-x) or Virtual Machine Control Blocks (AMD-V).

1.7.1 Architecture diagram



1.8. Types of Virtualization

1.8.1 Overview

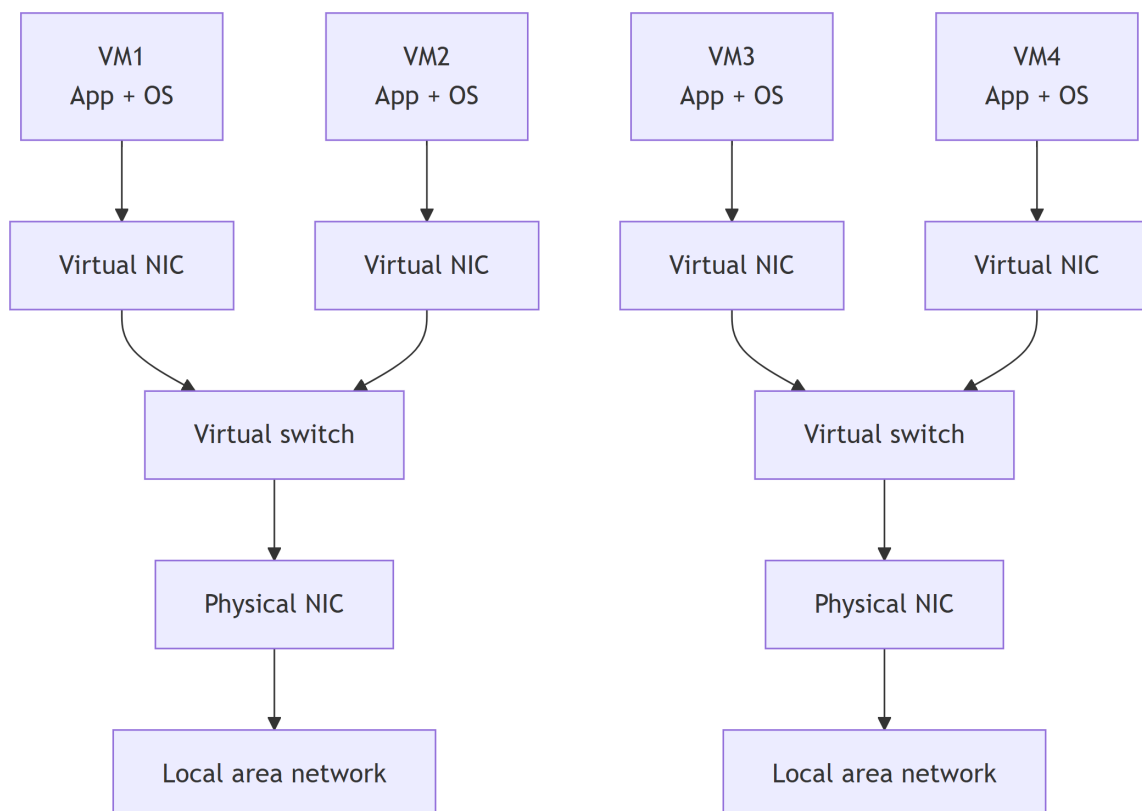


1.8.2 Desktop virtualization

Also called client virtualization. The virtualization happens on the user's site: desktops are replaced with thin clients that use datacentre resources.

1.8.3 Network virtualization

Part of the virtualization infrastructure, used especially when virtualizing servers. It allows multiple switches, VLANs and NAT-ing to be created in software.

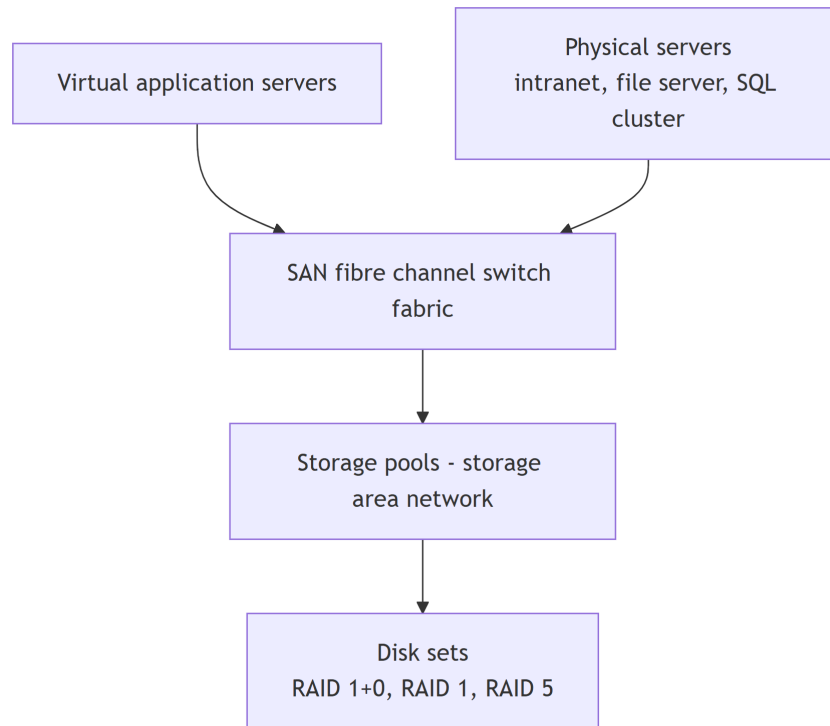


1.8.4 Server and machine virtualization

Virtualizing the server infrastructure, so no further physical servers are needed for different purposes.

1.8.5 Storage virtualization

Widely used in datacentres with large storage. It helps create, delete and allocate storage to different hardware, over a network connection. The leading technology here is the SAN.



1.8.6 Operating system level virtualization

An abstraction layer between the traditional OS and user applications. OS-level virtualization creates isolated **containers** on a single physical server, using one OS instance to utilize the hardware and software in data centres. The containers behave like real servers. It is commonly used to create virtual hosting environments that allocate hardware resources among a large number of mutually distrusting users, and to a lesser extent to consolidate server hardware by moving services on separate hosts into containers or VMs on one server.

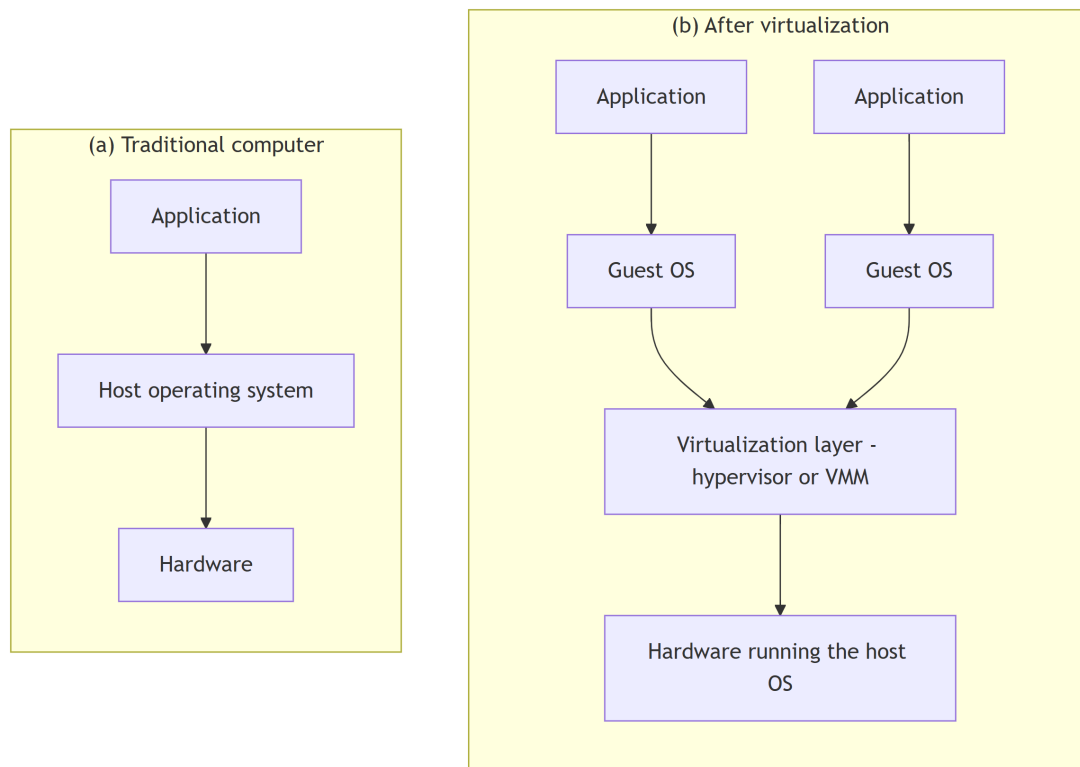
1.8.7 Application virtualization

The technology isolates applications from the underlying operating system and from other applications, to increase compatibility and manageability. Docker can be used for this purpose.

1.9. Levels of Virtualization

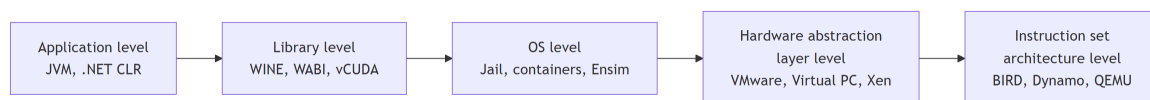
Short description: A traditional computer runs a host operating system tailored to its hardware architecture. After virtualization, different user applications managed by their own guest operating systems can run on the same hardware, independent of the host OS. This is done by adding a software layer called the virtualization layer, known as the hypervisor or virtual machine monitor.

1.9.1 Before and after virtualization



1.9.2 Implementation levels

Virtualization can be implemented at different levels within a computing system, offering varying degrees of isolation and flexibility.



1.9.3 Instruction set architecture level

- Virtualization is performed by emulating a given ISA with the ISA of the host machine.
- For example, MIPS binary code can run on an x86-based host machine through ISA emulation.
- It becomes possible to run a large amount of legacy binary code written for various processors on any given new hardware host.

1.9.4 Hardware abstraction level

- Hardware-level virtualization is performed right on top of the bare hardware. It generates a virtual hardware environment for a VM, and the process manages the underlying hardware through virtualization.
- The idea is to virtualize a computer's resources, such as its processors, memory and I/O devices, to upgrade the hardware utilization rate by multiple concurrent users.
- It was implemented in the IBM VM/370 in the 1960s. More recently the Xen hypervisor has been applied to virtualize x86-based machines running Linux or other guest OS applications.

1.9.5 Operating system level

- An abstraction layer between the traditional OS and user applications.
- It creates isolated containers on a single physical server, with the OS instances utilizing the hardware and software in data centres.
- The containers behave like real servers, and are commonly used for virtual hosting environments allocating hardware among mutually distrusting users.

1.9.6 Library support level

- Most applications use APIs exported by user-level libraries rather than lengthy system calls, so a well-documented API becomes another candidate for virtualization.
- Virtualization with library interfaces controls the communication link between applications and the rest of the system through API hooks.
- WINE implements this approach to support Windows applications on top of UNIX hosts.
- vCUDA allows applications executing within VMs to leverage GPU hardware acceleration.

1.9.7 User-application level

- Virtualization at the application level virtualizes an application as a VM. On a traditional OS an application runs as a process, so this is also known as process-level virtualization.
- The most popular approach is to deploy high level language (HLL) VMs. The virtualization layer sits as an application program on top of the operating system and exports the abstraction of a VM that runs programs compiled to a particular abstract machine definition.
- The Microsoft .NET CLR and the Java Virtual Machine are two good examples.

1.10. Containers

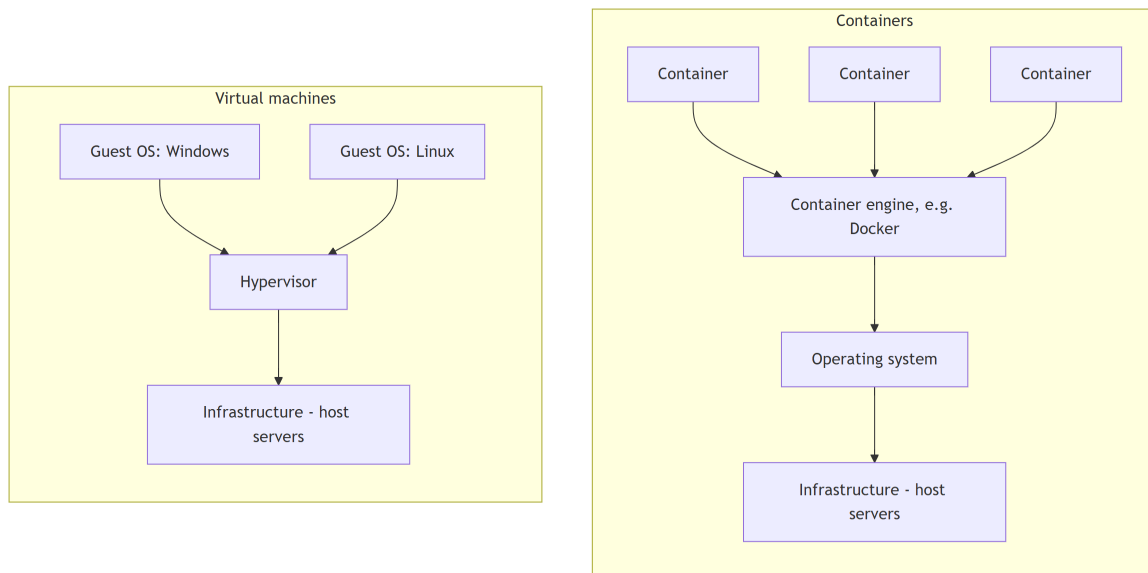
Short description: A container is an isolated, stand-alone unit that encapsulates an application and all its dependencies. It runs consistently in any environment, independent of the host system.

Theory: It is a light, executable software package that wraps everything an application needs: libraries, configuration files and binaries. The application behaves the same way on a developer's laptop, in a testing environment, or on a production server. Unlike traditional virtual machines that carry a full OS, containers pack only what is required, which makes them faster and more efficient.

How it works: Containerization is the process of packing an application together with all its dependencies into a container so it runs consistently from one computing environment to another. In simple terms it uses the host OS kernel to run many isolated instances of applications on the same machine, which makes it very lightweight and efficient.

1.10.1 Containers versus virtual machines

Both containers and VMs run applications in isolated environments, but they differ in architecture, performance, resource usage and startup time.



1.10.2 Comparison

	Containers	Virtual machines
Architecture	All containers share the host OS kernel; the running user spaces are isolated, which makes them lightweight.	A hypervisor running on the host OS includes a full guest OS with virtualized hardware.
Boot time	Much less, typically seconds, as no full OS has to boot.	Longer, because the full OS in the VM must be initialized.
Isolation	At the process level, less strong than a VM, though for many use cases this does not matter.	Very good, because each VM is a system on its own with its own OS.
Resource usage	Fewer resources: only the necessary binaries and libraries, not an entire OS.	Very high, as the full OS overhead is incurred for each instance.

1.11. Container Orchestration

Short description: Container orchestration is the process of automating the deployment, management, scaling and networking of containers throughout their lifecycle.

Theory: Orchestration platforms such as Kubernetes create a smooth, self-managing operation and coordinate microservices held in separate containers. Kubernetes eliminates many of the manual processes involved in deploying and scaling containerized applications, and Red Hat creates essential tools for securing, simplifying and automatically updating container infrastructure.

1.11.1 Why it is needed

When many containers run across different machines, managing them manually becomes difficult. An orchestration platform automates:

- **Deployment:** starts and distributes containers across servers.
- **Scaling:** increases or decreases the number of container instances based on demand.
- **Load balancing:** distributes incoming traffic among healthy containers.
- **Self-healing:** restarts failed containers or moves them to healthy nodes.
- **Service discovery:** allows containers to find and communicate with each other.
- **Rolling updates and rollbacks:** updates applications with minimal downtime and reverts changes if needed.
- **Resource management:** efficiently allocates CPU, memory and storage.

Benefits

Orchestration platforms automate the entire lifecycle of containers, transforming what would be a non-stop maintenance operation for IT teams into a smooth, self-managing one, and bring advantages in development methods, costs and security.

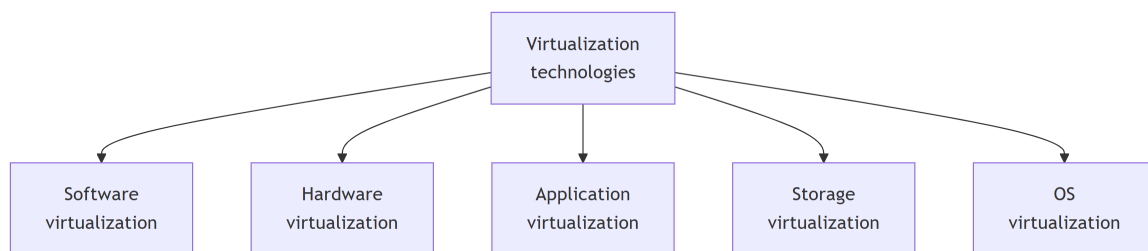
Unit 2: Virtualization Technologies and Data Center Management

2.1. Virtualization Technologies

Short description: Unit 1 covered what virtualization is and how a hypervisor works. This unit covers the technologies that implement it, the data centre those technologies run in, and the addressing scheme that ties the machines together.

Five technologies are studied in this unit.

1. Software virtualization
2. Hardware virtualization
3. Application virtualization
4. Storage virtualization
5. Operating system virtualization



2.2. Software Virtualization

Short description: Software virtualization is a technique that allows a server to run multiple operating systems and applications by creating a virtual environment.

Theory: Software virtualization develops a virtual environment in which software, applications and operating systems can be installed. A virtualized software is simply an application that has been installed inside such an environment. It lets a user run more than one operating system on the same machine. VMware and VirtualBox are common examples.

2.2.1 Components

- **Hypervisor:** a piece of software that manages and allocates hardware resources to the virtual machines.
- **Virtual machines:** independent computers, each running its own operating system on a portion of the physical machine's hardware.
- **Abstraction layer:** the layer of separation between the physical hardware and the virtual machines.

2.2.2 Types of software virtualization

The three forms studied in this unit are operating system virtualization, application virtualization and service virtualization. Each is covered in its own section below.

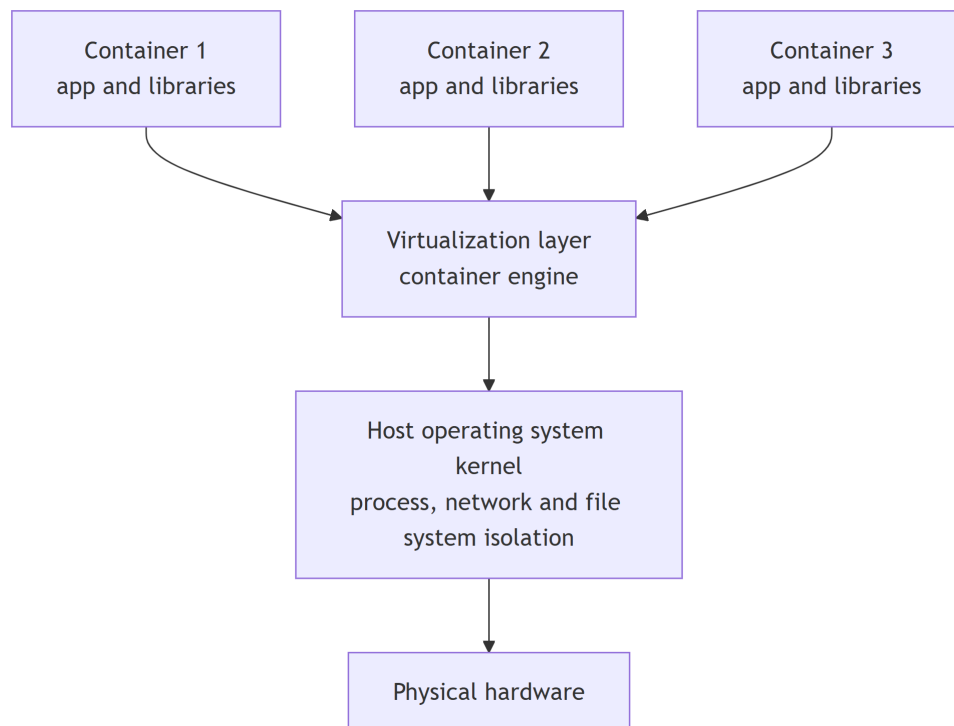
2.3. Operating System Virtualization

Short description: A virtualization technique in which the virtualization software runs on top of a host operating system and creates isolated environments called containers.

Theory: Multiple applications or services run independently on the same physical hardware while sharing the underlying OS kernel. The virtualization layer creates an abstraction between the hardware and the applications, which improves resource utilization and isolation. Unlike traditional virtualization, which creates complete virtual machines, OS-based virtualization shares the host kernel among the containers, so it is lighter and more efficient.

How it works: The host operating system kernel manages the containers through specialised features that control resource access and allocation. Each container operates as an isolated environment with its own process space, network interfaces and file systems, while sharing the underlying kernel. The kernel enforces limits on CPU time, memory usage, disk space and network bandwidth for each container, so one container cannot affect the others and no conflicts or security breaches arise between them.

2.3.1 Architecture diagram



2.3.2 Types of virtual disk

- **Private virtual disk:** dedicated to a single container. It keeps data and configuration across restarts, much like a local hard disk.
- **Shared virtual disk:** used by several containers at the same time. Changes go to a temporary cache that is cleared on restart, which gives every container the same baseline configuration.

2.3.3 Advantages

- **Resource efficiency:** containers share the host kernel, so memory and storage overhead is lower than with full virtualization.
- **Fast startup:** a container starts quickly because no full OS boot is needed.
- **High density:** more containers fit on the same hardware than virtual machines.
- **Portability:** containers move easily between host systems.

2.3.4 Disadvantages

- **OS dependency:** every container must use the same operating system as the host.
- **Security concerns:** a kernel vulnerability can affect all containers at once.
- **Limited isolation:** isolation is weaker than in hardware-based virtualization.

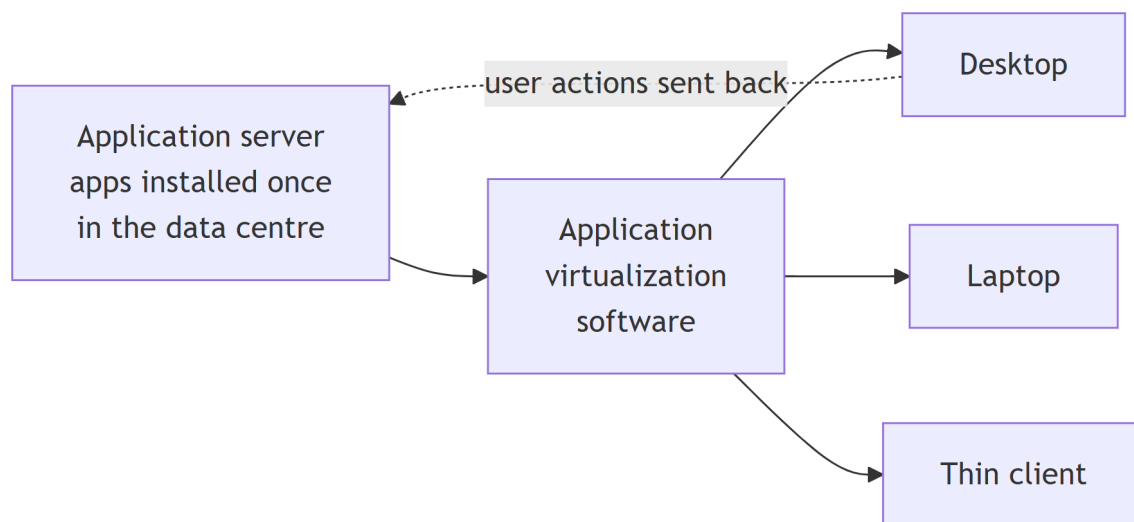
2.4. Application Virtualization

Short description: Application virtualization is the ability to run an application in an environment different from the one it was developed for.

Theory: It allows IT administrators to install applications on a server and then deliver them remotely to users' computers. The experience of using a virtualized application is the same as using an application installed on a physical machine.

How it works: The common approach is server based. An administrator installs the applications once, either in the organisation's data centre or through a hosting service, and then uses application virtualization software to deliver them to users' desktops and other connected devices. The user accesses the application as if it were installed locally, and the user's actions are conveyed back to the server to be executed. Application virtualization is an integral part of digital workspaces and desktop virtualization.

2.4.1 Architecture diagram



2.4.2 Benefits

- **Simplified management:** the applications to be delivered are installed on a single server rather than on every user device, which makes installing software, updates and patches far easier for the IT team.
- **Scalability:** administrators can deploy virtual applications to every connected device regardless of its operating system or storage capacity. The organisation then spends less on hardware, because employees only need a basic computer to reach the applications they work with.
- **Security:** access to applications is controlled centrally, so licences are easy to revoke when a user no longer needs them. The application never has to be uninstalled from a user's device, because it stays resident but reachable only through its container. This central control matters most when a device is lost or stolen, since sensitive data can no longer be reached remotely.

2.5. Service Virtualization

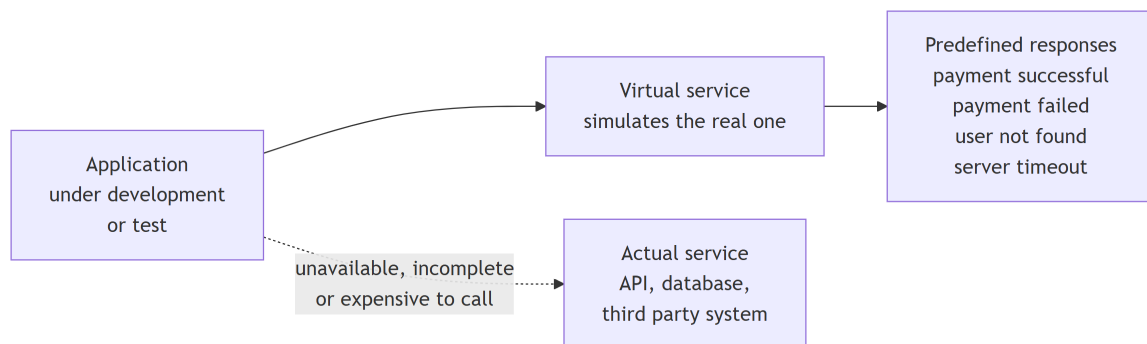
Short description: Service virtualization simulates the behaviour of software services, APIs, databases or third-party applications.

Theory: It lets developers and testers build, integrate and test an application even when the real service is unavailable, incomplete, expensive to access or difficult to configure. In simple terms, it creates a virtual version of a dependent service that behaves like the real one without requiring the actual system.

How it works: Instead of communicating with the actual service, the application talks to the virtual service, which returns predefined or dynamically generated responses.

Examples: Typical simulated responses are payment successful, payment failed, user not found, server timeout and invalid request.

2.5.1 Architecture diagram



2.6. Hardware Virtualization

Short description: Hardware virtualization allows multiple virtual machines to run on a single physical machine.

Theory: It creates a virtual version of a computer's hardware components: the CPU, memory, storage and network. The resulting virtual environment, the virtual machine, acts like a real computer and runs its own operating system and applications independently of the host system.

How it works: A software layer called the hypervisor creates and manages the virtual machines on a single physical server. The hypervisor allocates the server's resources, such as CPU, memory and storage, to each VM, so several operating systems and applications run independently on the same hardware.

2.6.1 Benefits

- **Improved resource utilization:** consolidating several workloads onto one physical server makes better use of the hardware, so fewer physical servers are needed and energy consumption and floor space both drop.
- **Cost savings:** fewer machines means savings on both capital expenditure, which is buying the servers, and operational expenditure, which is maintaining and powering them.
- **Flexibility and scalability:** virtual machines are easy to create, modify and move, which matters for a business that has to adapt quickly to changing workloads.
- **Isolation and security:** each VM operates in isolation, so malware or a system failure is contained within a single VM.
- **Simplified management:** centralised tools let administrators manage many virtual environments from one interface, which is especially useful when the infrastructure is spread across several locations.

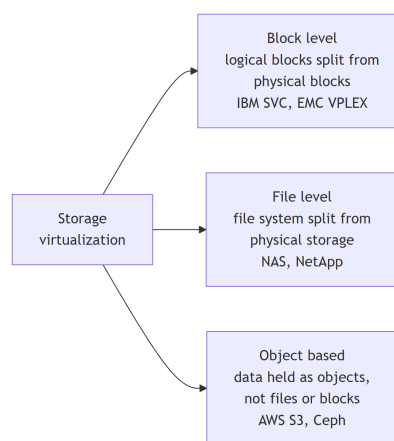
2.7. Storage Virtualization

Short description: Storage virtualization brings several storage devices together into a single entity, for better efficiency, security and management.

Theory: Storage virtualization is a newer concept under virtualization: storage systems use it to gain functionality and features. A storage system is also called a storage array, a disk array or a filer. It uses special hardware and software to provide fast and reliable storage for computing and processing data, and it is designed to accommodate large capacity together with data protection.

How it works: Multiple physical storage devices are combined into a single logical storage pool by a virtualization layer. That layer manages allocation, maps logical volumes to physical devices and handles every read and write transparently. Users and applications see a single logical storage system, while the software manages the physical storage underneath and improves utilization, scalability, performance and ease of management.

2.7.1 Types of storage virtualization



- **Block level virtualization:** separates logical storage from physical storage at the block level, as in IBM SVC and EMC VPLEX.
- **File level virtualization:** abstracts the file system from the physical storage, as in NAS solutions such as NetApp.
- **Object based storage:** holds data as objects rather than as files or blocks, as in AWS S3 and Ceph.

2.7.2 Benefits

- **Enhanced storage utilization:** aggregates different storage devices efficiently.
- **Scalability:** storage scales easily as requirements grow.
- **Savings:** hardware cost falls because existing resources are reused.
- **Very high availability:** redundancy and failover support are built in.
- **Improved performance:** workload is distributed dynamically across the storage.
- **Easier data migration:** data moves from one storage system to another seamlessly.
- **Energy efficiency:** optimised storage usage draws less power.

2.8. The Data Center

Short description: A data centre is a centralised facility that houses computing resources: servers, storage systems, networking equipment, power systems, cooling systems and security infrastructure.

Theory: It is designed to store, process, manage and distribute large amounts of data and applications continuously and securely. In simple terms, the data centre is the backbone of modern IT infrastructure, providing the environment needed to run business applications, websites, cloud services, databases and enterprise systems.

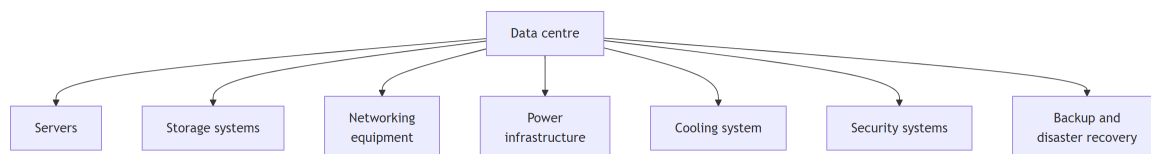
2.8.1 Need for a data centre

- Centralised storage and management of data
- High-speed data processing
- Secure storage of business information
- Continuous availability of services
- Disaster recovery and backup
- Support for cloud computing and virtualization
- Efficient resource utilization

2.8.2 Objectives of a data centre

- Provide reliable IT services
- Ensure high availability of applications
- Store and process large volumes of data
- Protect data against failures and cyber threats
- Improve business continuity
- Optimise resource utilization

2.8.3 Components

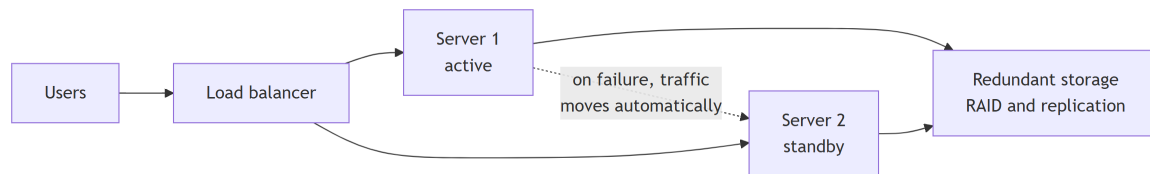


2.9. High Availability of a Data Center

Short description: High availability is the capability of a data centre to keep applications, servers, storage and network services available with minimal or no downtime, even when hardware, software or network failures occur.

Theory: The primary objective is continuous service to users, achieved by eliminating single points of failure and by enabling automatic recovery from failures.

2.9.1 Architecture diagram



2.9.2 Need for high availability

Organisations require uninterrupted service, for the following reasons.

- Continuous business operations
- Better customer satisfaction
- Prevention of financial loss
- Protection against hardware failures
- Data protection
- Disaster recovery
- Compliance with service level agreements
- Improved business reputation

2.9.3 Causes of data centre downtime

Category	Typical causes
Hardware failures	Server crash, hard disk failure, storage controller failure, memory failure, power supply unit failure
Software failures	Operating system crash, application bugs, database corruption, configuration errors
Network failures	Router failure, switch failure, ISP outage, cable damage
Human errors	Wrong configuration, accidental deletion, improper maintenance
Power failures	Power outage, UPS failure, generator malfunction

2.10. Data Center Maintenance

Short description: Data centre maintenance is the set of processes that keep a data centre operating as required, for example to meet an uptime objective such as 99.9 per cent or a service level agreement.

Theory: The tasks are monitoring and inspecting the equipment and systems, from hardware to electrical, cooling and cabling, and then carrying out the maintenance and repair work needed to meet the operational objectives.

2.10.1 Why it matters

Maintenance underpins most of the IT services a business runs. If the business runs its ERP software in its own data centre, maintenance is what keeps the ERP working. Maintenance is also cheaper than the alternative: system failures are stressful and costly to handle, and proactive work detects problems before they cause trouble.

The following problems arise from insufficient or unplanned maintenance.

- **Equipment failures:** neglected cooling leads to hot spots, which cause equipment to overheat and fail.
- **Power outages:** poor maintenance of the power system causes an outage, which disrupts service and can even lose data.
- **Cabling problems:** badly set up and managed cables lead to unintended disconnections.
- **Water damage:** ignoring water lines and temperature changes leads to flooding, which makes equipment fail.
- **Cleaning problems:** accumulated dust and debris interrupt air flow, damage sensitive equipment and can even start fires.

2.11. Planned Maintenance

Short description: Planned maintenance is maintenance scheduled in advance and carried out during predefined maintenance windows, so that disruption to users and business operations is minimal.

Theory: Because the work is planned, users are informed beforehand, backups are taken and contingency plans are prepared.

2.11.1 Objectives

- Improve system performance
- Prevent equipment failure
- Increase system reliability
- Enhance security
- Extend equipment lifespan
- Ensure business continuity

2.11.2 Characteristics

- Scheduled in advance
- Performed during maintenance windows
- Users are notified before the work starts
- Backup systems are prepared
- Low risk of unexpected failure
- Easy to manage and monitor

2.11.3 Activities

Area	Activities
Hardware	Cleaning servers and racks, replacing faulty hard disks, upgrading RAM, replacing power supplies and cooling fans, checking UPS batteries, generator maintenance. For example, replacing an aging hard disk before it fails.
Software	Operating system updates, security patches, application and database upgrades, firmware and driver updates. For example, applying the latest security patches to Linux servers.
Network	Router firmware updates, switch software upgrades, firewall configuration updates, cable inspection, network optimisation. For example, upgrading switch firmware for stability.
Storage	RAID health check, storage expansion, disk replacement, SAN firmware update, backup verification.
Security	Antivirus updates, firewall rule updates, access control review, password policy updates, security auditing.
Backup	Testing backups, verifying recovery procedures, restoring sample data, disaster recovery testing.
Cooling	Cleaning air filters, checking cooling units, temperature monitoring, airflow optimisation.
Power	UPS testing, generator testing, battery replacement, PDU inspection.

2.11.4 Advantages

- Reduces unexpected failures
- Improves system availability
- Enhances security
- Increases equipment life
- Reduces repair costs
- Better resource planning
- Minimises business disruption

2.11.5 Disadvantages

- Requires scheduled downtime, unless redundant systems are in place
- Needs careful planning
- May temporarily affect users
- Requires extra manpower and coordination

2.12. Unplanned Maintenance

Short description: Unplanned maintenance is unexpected maintenance caused by a sudden failure, fault or emergency. It is not scheduled in advance and needs immediate action to restore normal operation and keep downtime short.

2.12.1 Causes

- Hardware failures in servers, storage devices or network switches
- Power outages or UPS failure
- Cooling system malfunction
- Network connectivity issues
- Software crashes or operating system failures
- Cyberattacks such as malware, ransomware and DDoS
- Human errors such as accidental deletion or wrong configuration
- Fire, flood or other natural disasters

2.12.2 Common activities

1. **Emergency hardware replacement:** replace failed hard disks, SSDs, RAM, CPUs, power supplies or network cards, and faulty switches, routers or cables.
2. **Server recovery:** reboot crashed servers, restore failed virtual machines and recover operating systems from backup images.
3. **Storage recovery:** replace failed disks in RAID arrays, rebuild the RAID configuration and restore lost data from backups.
4. **Network troubleshooting:** find and repair broken links, replace damaged switches or routers and reconfigure network settings after a failure.
5. **Power system recovery:** repair or replace faulty UPS batteries, restore generator functionality and switch to backup power during a utility failure.
6. **Cooling system repair:** fix malfunctioning air conditioning units, restore temperature and humidity control and prevent servers from overheating.

2.12.3 Steps followed

1. Detect the problem through the monitoring systems.
2. Analyse and identify the root cause.
3. Isolate the affected equipment or service.
4. Repair or replace the faulty components.
5. Restore systems and services.
6. Test functionality to confirm normal operation.
7. Document the incident and the corrective actions.
8. Implement preventive measures so it does not recur.

2.12.4 Challenges

- Unexpected downtime and loss of productivity
- Risk of data loss
- Increased operational costs
- Service level agreement violations and customer dissatisfaction
- Higher workload for the IT staff

2.12.5 Best practices to reduce it

- Implement continuous monitoring and alerting.
- Perform regular preventive maintenance.
- Maintain redundant hardware and network paths.
- Keep reliable backups and disaster recovery plans.
- Use high availability and failover systems.
- Conduct periodic hardware health checks.
- Apply security updates and patches on time.

2.12.6 Planned versus unplanned maintenance

Planned maintenance	Unplanned maintenance
Scheduled in advance	Occurs unexpectedly
Preventive in nature	Corrective or emergency in nature
Minimal downtime	Downtime is often unavoidable
Lower maintenance cost	Higher maintenance cost
Better resource planning	Immediate resource allocation required
Reduces the risk of failures	Responds after failures occur

2.13. Server Migration

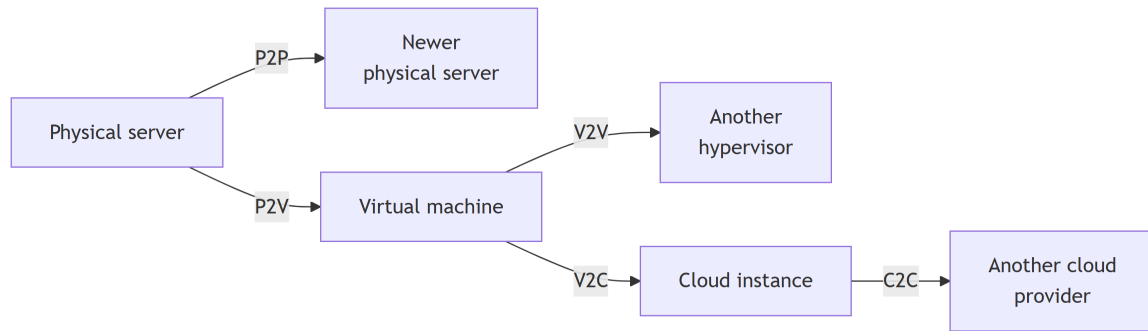
Short description: Server migration is the process of transferring a server's data, processes and configuration to a new target server or cloud instance.

Theory: Organisations migrate servers for resource optimisation, to reduce maintenance overhead, for better support and for deeper integration and modernisation. The process varies with the source and target machine architectures, and it needs careful planning, the right tools and testing to succeed.

2.13.1 Benefits

- **Update to modern services:** move from outdated and restrictive infrastructure to modern, scalable, maintainable platforms. In a cloud environment the servers are virtualized and are called instances.
- **Integration with other services:** modern server environments connect more easily to well-known tools and services, which enables faster automation.
- **Lower management overhead:** migrating to the cloud shifts maintenance from the organisation's own IT team to the hosting provider. On-premises hardware means the internal team handles updates and backups; on cloud infrastructure the provider does.
- **Cost reductions:** on-premises infrastructure is a capital expenditure, while cloud services are billed monthly, annually or pay as you go, which for many organisations is the more efficient budgeting model.
- **Security hardening:** modern platforms carry the latest encryption, identity controls and data security tools, although configuring the environment to suit the organisation's own requirements remains the customer's job.
- **Better performance:** cloud instances reach data held in the same cloud environment faster, and offer a wider range of storage and compute options.

2.13.2 Types of server migration



- **Physical to physical (P2P):** moving a workload from one physical server to another, for example replacing an old server with a newer, faster one.
- **Physical to virtual (P2V):** converting a physical server into a virtual machine. This gives better resource utilization, easier backup, lower hardware cost and simpler management.
- **Virtual to virtual (V2V):** migrating a VM from one virtualization platform to another, for example VMware to Hyper-V or VMware ESXi to KVM.
- **Virtual to cloud (V2C):** moving virtual machines from on-premises infrastructure to a cloud service.
- **Cloud to cloud (C2C):** migrating applications and data between cloud providers, for example AWS to Microsoft Azure or Azure to Google Cloud Platform.

2.13.3 Migration considerations

Business considerations

Migration should align with the business goals. The points to weigh are business continuity, downtime requirements, budget constraints, return on investment, project timeline and regulatory compliance. For example, a hospital migrating its patient databases must keep medical records reachable throughout the migration.

Hardware considerations

Verify that the destination environment meets the hardware requirements: processor architecture, number of CPU cores, RAM capacity, storage capacity, disk type (SSD or HDD) and network interface speed.

Software compatibility

Applications must still work after the move. Check operating system compatibility, database versions, middleware compatibility, drivers and application dependencies.

Data integrity

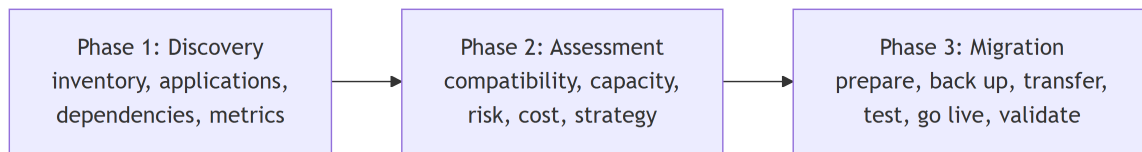
Data must stay accurate and complete throughout the migration. The activities are a full data backup, data validation, checksum verification and database consistency checks.

Self study

Network considerations, security considerations, performance considerations, downtime planning and risk management are left as self study.

2.14. Planning a Server Migration

A successful migration begins with careful planning. The pre-migration process has three phases: discovery, assessment and migration planning.



2.14.1 Phase 1: Discovery

Discovery is the process of collecting complete information about the existing IT infrastructure before the migration. Its purpose is to understand the current environment and identify every resource that has to move.

1. **Server inventory:** server name, IP address, operating system, CPU details, RAM, storage and installed software.
2. **Application discovery:** installed applications, running services, web servers, databases and middleware.
3. **Dependency mapping:** applications rarely work on their own, so identify the dependencies among applications, databases, file servers, authentication servers and external APIs.
4. **Performance monitoring:** CPU utilization, RAM usage, disk usage, network traffic, peak workload and storage growth. These metrics are what size the new environment correctly.
5. **Security discovery:** user accounts, access rights, certificates, firewall rules and encryption methods.

2.14.2 Phase 2: Assessment

Assessment analyses the information collected during discovery, to judge whether the migration is feasible and to prepare a strategy. Its objectives are to evaluate compatibility, estimate resources, identify risks, estimate costs and select the migration method.

1. **Compatibility assessment:** operating system support, application compatibility, database compatibility, driver support and hardware compatibility.
2. **Capacity planning:** estimate future CPU, memory, storage and network bandwidth requirements, so the new environment can carry both the current and the future workload.
3. **Risk assessment:** analyse downtime, performance degradation, security issues, data corruption and hardware failure, and assign a probability and an impact to each.
4. **Cost assessment:** hardware, cloud services, licensing, migration tools, staff training and maintenance.
5. **Migration strategy selection:** covered below.
6. **Backup planning:** take complete backups, verify their integrity, store them securely and prepare rollback procedures.

Migration strategies

Strategy	What it means	Notes
Rehosting	Move applications without changing their architecture, also called lift and shift.	Fast, low cost, minimal modifications.
Replatforming	Make minor optimisations while migrating.	For example, moving a database to a managed cloud database service.
Refactoring	Redesign the application for a cloud-native environment, also called re-architecting.	High scalability, better performance and long-term flexibility, but high cost and complexity.
Repurchasing	Replace the existing software with a SaaS product.	For example, replacing an on-premises email server with Microsoft 365.

2.14.3 Phase 3: Migration

Migration is the actual transfer of applications, operating systems, databases and data to the target environment.

1. **Prepare the target environment:** configure servers, virtual machines, storage, networks and security policies.
2. **Back up the existing systems:** operating system, applications, databases, configuration files and user data.
3. **Transfer the data:** by network transfer, storage replication, backup restoration, migration tools or cloud migration services.
4. **Migrate the applications:** install the applications, restore the databases, configure the services and update the configuration files.
5. **Test:** functional testing, performance testing, security testing and user acceptance testing, to confirm the applications work after the move.
6. **Go live:** redirect user traffic, update DNS records, activate production services and monitor system health.
7. **Post-migration validation:** verify data integrity, application functionality, user authentication, network connectivity, backup operations and performance.

2.14.4 Best practices

- Maintain a complete inventory of servers and applications.
- Perform full backups and verify their integrity.
- Test the migration in a staging environment before production.
- Schedule the migration during off-peak hours to minimise downtime.
- Monitor system performance throughout the migration.
- Prepare and test a rollback plan.
- Validate data integrity and application functionality afterwards.
- Document every step for future reference and audits.

2.14.5 Advantages of proper planning

Reduced downtime, improved application performance, better scalability, efficient resource utilization, lower operational costs, enhanced security, improved disaster recovery and simpler infrastructure management.

2.15. Networking Concepts: Masks and CIDR

Short description: A mask is what separates the network identifier from the host identifier in an IP address.

Theory: A mask is a 32-bit number made of a run of 1s followed by a run of 0s. It does not apply to classes D and E. Classless interdomain routing, or CIDR, writes the mask in the short form /*n*, where *n* is the number of 1 bits.

CIDR	Dotted decimal	Binary				Class
/8	255.0.0.0	11111111	00000000	00000000	00000000	A
/16	255.255.0.0	11111111	11111111	00000000	00000000	B
/24	255.255.255.0	11111111	11111111	11111111	00000000	C

Notes

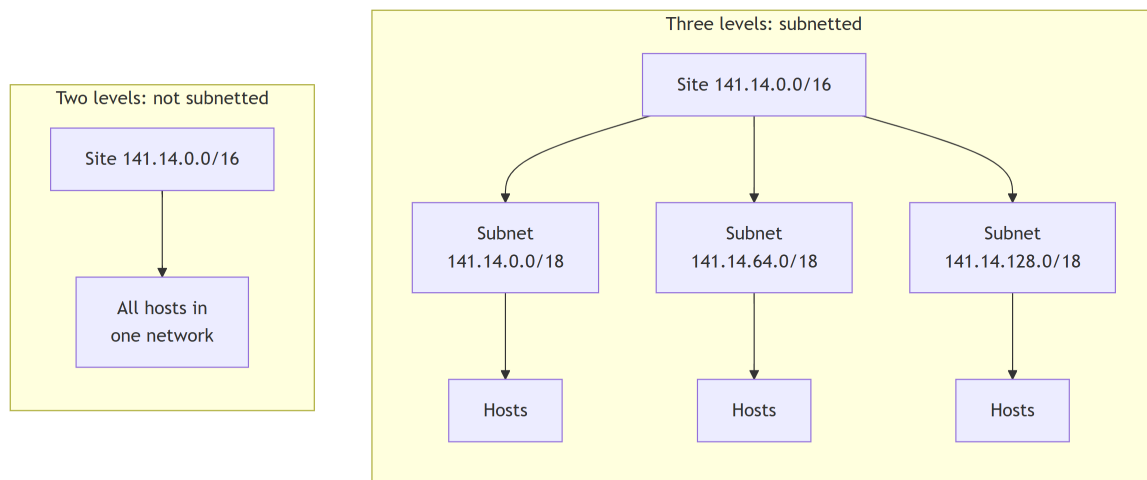
- **Subnetting:** an organisation granted a large block in class A or B divides the addresses into several contiguous groups and assigns each group to a smaller network, called a subnet.
- **Supernetting:** an organisation combines several class C blocks to create a larger range of addresses. Several networks are combined into one supernet, or supernet.

2.16. Subnetting

Short description: Subnetting divides a large granted block into several smaller networks, by borrowing bits from the host part and adding them to the network part.

Theory: Subnetting increases the number of 1s in the mask: a subnet mask has more 1s than the corresponding default mask. To build a subnet mask, change some of the leftmost 0s in the default mask to 1s. The number of subnets follows from the number of extra 1s: if there are n extra 1s there are 2^n subnets, and if N subnets are needed the number of extra 1s is $\log_2 N$. The number of subnets must therefore be a power of 2.

2.16.1 Levels of hierarchy



Without subnetting the address has two levels of hierarchy, network and host. With subnetting it has three: network, subnet and host.

2.16.2 Finding the subnetwork address

The subnet address is found the same way as the network address, by applying the mask to the address. There are two ways to do it.

- **Straight method:** write both the address and the mask in binary and apply the AND operation.
- **Short-cut method:** if the byte in the mask is 255, copy the byte of the address. If the byte in the mask is 0, replace the byte of the address with 0. If the byte is neither 255 nor 0, write that byte of the mask and of the address in binary and AND them.

Example: straight method

What is the subnetwork address if the destination address is 200.45.34.56 and the subnet mask is 255.255.240.0?

Line	Binary
Address	11001000 00101101 00100010 00111000
Mask	11111111 11111111 11110000 00000000
Result	11001000 00101101 00100000 00000000

The subnetwork address is 200.45.32.0.

Example: short-cut method

What is the subnetwork address if the destination address is 19.30.84.5 and the mask is 255.255.192.0? The first two bytes of the mask are 255, so 19 and 30 are copied. The last byte of the mask is 0, so 5 becomes 0. The third byte is neither, so AND it: 01010100 AND 11000000 is 01000000, which is 64. The subnetwork address is 19.30.64.0.

Example: designing subnets in class C

A company is granted the site address 201.70.64.0, a class C address, and needs six subnets.

The default mask of class C has 24 ones. Six is not a power of 2, and the next power of 2 is $8 = 2^3$, so three more 1s are needed. The subnet mask has $24 + 3 = 27$ ones and $32 - 27 = 5$ zeros, which is 11111111 11111111 11111111 11100000, or 255.255.255.224. There are 8 subnets and each holds $2^5 = 32$ addresses.

Example: designing subnets in class B

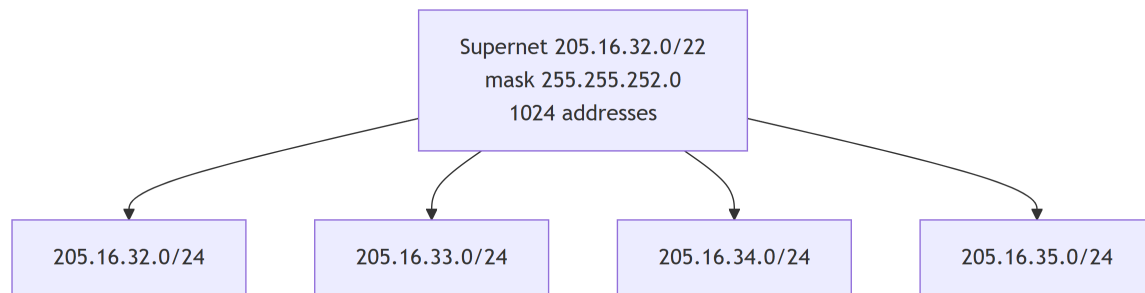
A company is granted the site address 181.56.0.0, a class B address, and needs 1000 subnets.

The default mask of class B has 16 ones. One thousand is not a power of 2, and the next power of 2 is $1024 = 2^{10}$, so ten more 1s are needed. The subnet mask has $16 + 10 = 26$ ones and $32 - 26 = 6$ zeros, which is 11111111 11111111 11111111 11000000, or 255.255.255.192. There are 1024 subnets and each holds $2^6 = 64$ addresses.

2.17. Supernetting

Short description: Supernetting is the opposite of subnetting. Bits are borrowed from the network side to combine a group of networks into one large supernetwork.

Theory: Demand for midsize blocks is high. Class A and class B addresses are almost depleted, while class C addresses are still available but a block of 256 addresses is too small for many organisations. In supernetting, several class C blocks are combined into a larger range. This decreases the number of 1s in the mask: for four class C blocks the mask changes from /24 to /22. An organisation needing 1000 addresses can be granted four contiguous class C blocks forming one supernetwork.



In subnetting, the first address of the subnet and the subnet mask define the range of addresses. In supernetting, the first address of the supernet and the supernet mask define the range.

Example: which packet belongs to the supernet

A supernet has the first address 205.16.32.0 and the supernet mask 255.255.248.0. A router receives three packets, for 205.16.37.44, 205.16.42.56 and 205.17.33.76.

Applying the supernet mask gives 205.16.32.0, 205.16.40.0 and 205.17.32.0 respectively. Only the first address belongs to this supernet.

Example: valid sets of blocks

A company needs 600 addresses. Which of these sets of class C blocks can form a supernet?

1. 198.47.32.0, 198.47.33.0, 198.47.34.0. No, there are only three blocks.
2. 198.47.32.0, 198.47.42.0, 198.47.52.0, 198.47.62.0. No, the blocks are not contiguous.
3. 198.47.31.0, 198.47.32.0, 198.47.33.0, 198.47.52.0. No, 31 in the first block is not divisible by 4.
4. 198.47.32.0, 198.47.33.0, 198.47.34.0, 198.47.35.0. Yes, all three requirements are fulfilled.

Example: the supernet mask for 16 blocks

Sixteen blocks need four 1s of the default mask changed to 0s, so the mask is 11111111 11111111 11110000 00000000, or 255.255.240.0.

Example: size and range of a supernet

A supernet has the first address 205.16.32.0 and the supernet mask 255.255.248.0. The mask has 21 ones and the default mask has 24, so the difference of 3 gives $2^3 = 8$ blocks. They run from 205.16.32.0 to 205.16.39.0, so the first address is 205.16.32.0 and the last is 205.16.39.255.

2.18. Classless Addressing

Short description: Classless addressing removes the classes altogether. Addresses are still granted in blocks, but the size of a block varies with the nature and size of the organisation.

Theory: Classless addressing was designed to overcome address depletion and give more organisations access to the Internet. A household may be given only two addresses, while a large organisation may be given thousands. It is referred to as classless interdomain routing, or CIDR, and the block granted is called a CIDR block.

2.18.1 Rules for allocating a block

1. The IP addresses in the CIDR block must all be contiguous.
2. The block size must be a power of 2, and the size of the block equals the number of IP addresses in it.
3. The first IP address of the block must be divisible by the block size.

For the beginning address, if the block has fewer than 256 addresses only the rightmost byte has to be checked; if it has fewer than 65,536 addresses only the two rightmost bytes have to be checked, and so on.

2.18.2 Slash notation

Slash notation, also called CIDR notation, is a shorter way of writing a subnet mask: instead of the full mask, only the number of 1 bits is written after a slash. A block in classes A, B and C is written A.B.C.D/n, where n is 8, 16 or 24 respectively.

2.18.3 Extracting information from an address

Given any address in the block and the prefix length n , three facts follow.

1. The number of addresses in the block is $N = 2^{32-n}$.
2. The first address is found by keeping the n leftmost bits and setting the $32 - n$ rightmost bits to 0.
3. The last address is found by keeping the n leftmost bits and setting the $32 - n$ rightmost bits to 1.

2.18.4 Address mask

The address mask is a 32-bit number whose n leftmost bits are 1 and whose remaining $32 - n$ bits are 0. A computer finds it easily because it is the complement of $2^{32-n} - 1$. Defined this way, the mask lets a program extract the block information with the bitwise NOT, AND and OR operations.

1. Number of addresses: $N = \text{NOT}(\text{mask}) + 1$.
2. First address: (any address in the block) AND (mask).
3. Last address: (any address in the block) OR NOT(mask).

Example: range of a block

A small organisation is given the block 205.16.37.24/29. To find the last address, keep the first 29 bits and change the last 3 bits to 1s.

Line	Binary
Beginning	11001111 00010000 00100101 00011000
Ending	11001111 00010000 00100101 00011111

There are 8 addresses in the block. The same answer follows from the suffix length: it is $32 - 29 = 3$, so there are $2^3 = 8$ addresses, and if the first address is 205.16.37.24 the last is 205.16.37.31, because $24 + 7 = 31$.

Example: finding the network address

What is the network address if one of the addresses is 167.199.170.82/27?

The prefix length is 27, so keep the first 27 bits and change the remaining 5 to 0s. Those 5 bits fall in the last byte, which is 01010010. Changing the last 5 bits to 0 gives 01000000, or 64. The network address is 167.199.170.64/27.

Example: subnetting a classless block

An organisation is granted 130.34.12.64/26 and needs four subnets.

The suffix length is 6, so the block holds $2^6 = 64$ addresses. Split four ways, each subnet has 16 addresses. Four subnets need two more 1s on the site prefix, so the subnet prefix is /28.

Subnet	Range
1	130.34.12.64/28 to 130.34.12.79/28
2	130.34.12.80/28 to 130.34.12.95/28
3	130.34.12.96/28 to 130.34.12.111/28
4	130.34.12.112/28 to 130.34.12.127/28

Example: first and last address of a block

One address of a block is 205.16.37.39/28. In binary it is 11001101 00010000 00100101 00100111.

Setting the $32 - 28 = 4$ rightmost bits to 0 gives 11001101 00010000 00100101 00100000, which is 205.16.37.32. Setting the same 4 bits to 1 gives 11001101 00010000 00100101 00101111, which is 205.16.37.47.

The same result comes from the mask, which for /28 is 11111111 11111111 11111111 11110000. ANDing the address with the mask gives the first address. ORing it with the complement of the mask gives the last address. Complementing the mask, reading it as a decimal number and adding 1 gives the number of addresses.

Example: an ISP distributing a block

An ISP is granted 190.100.0.0/16 and has to serve three groups of customers: 64 customers needing 256 addresses each, 128 customers needing 128 addresses each, and 128 customers needing 64 addresses each.

Group	Suffix	Prefix	Sub-blocks
1	8	/24	190.100.0.0/24 to 190.100.63.255/24, total $64 \times 256 = 16,384$
2	7	/25	190.100.64.0/25 to 190.100.127.255/25, total $128 \times 128 = 16,384$
3	6	/26	190.100.128.0/26 to 190.100.159.255/26, total $128 \times 64 = 8,192$

The block granted holds 65,536 addresses and 40,960 of them are allocated, so 24,576 addresses are still available.

2.19. Practice Problems

Short description: Twenty-two worked problems covering classful addressing, subnetting, classless (CIDR) blocks, VLSM and supernetting, in the style of the examples above. Each question is followed immediately by its full solution.

Question 1

Find the network address of the host with IP address 168.14.32.12, assuming the default classful mask.

Given

IP Address = 168.14.32.12

Step 1: Identify the Class

The first octet is **168**.

Class	Range
A	0–127
B	128–191
C	192–223

Since $128 \leq 168 \leq 191$, this is a **class B** address.

Step 2: Default Classful Mask

The class B default mask is:

255.255.0.0 $\rightarrow$ /16

This means the **first two octets** identify the network, and the **last two octets** identify the host.

Step 3: Apply the Mask (Binary AND)

Line	Binary
IP (168.14.32.12)	10101000 00001110 00100000 00001100
Mask (255.255.0.0)	11111111 11111111 00000000 00000000
AND (168.14.0.0)	10101000 00001110 00000000 00000000

Final Answer

Term	Value
Class	B
Default mask	255.255.0.0 (/16)
Network address	168.14.0.0

Question 2

Find the subnetwork address for IP address 200.45.34.56 and subnet mask 255.255.240.0, using the straight binary method.

Line	Binary
Address	11001000 00101101 00100010 00111000
Mask	11111111 11111111 11110000 00000000
Result	11001000 00101101 00100000 00000000

The mask has 20 ones, so the prefix is /20 and the **subnetwork address is 200.45.32.0**.

Question 3

Find the subnetwork address for IP address 19.30.84.5 and subnet mask 255.255.192.0, using the shortcut method.

The first two mask bytes are 255, so 19 and 30 are copied unchanged. The last byte is 0, so the fourth octet becomes 0. The third byte, 192, is neither 255 nor 0, so it must be worked out: the block size is $256 - 192 = 64$, and the multiples of 64 around the third octet (84) are 64–127, so 84 falls in the block starting at 64. The **subnetwork address is 19.30.64.0** (prefix /18).

Question 4

A router receives a packet with destination address 190.240.224.33. Find the subnetwork address if the subnet mask is /19.

/19 is 255.255.224.0. The mask breaks in the third octet (224), giving a block size of $256 - 224 = 32$. The third octet, 224, is itself an exact multiple of 32, so it is the start of its own block (224–255). The first two octets and the fourth octet follow the usual rule, giving the **subnetwork address 190.240.224.0**.

Line	Binary
Address	10111110 11110000 11100000 00100001
Mask	11111111 11111111 11100000 00000000
Result	10111110 11110000 11100000 00000000

Question 5

A router receives a packet with destination address 190.240.33.91. Show how the router finds the network address and the subnetwork address, assuming the subnet mask is /19.

This asks for two different addresses, found with two different masks.

Network address (classful): the first octet, 190, is class B, so the default mask is 255.255.0.0. ANDing gives 190.240.0.0.

Subnetwork address (/19): /19 is 255.255.224.0, which breaks in the third octet with a block size of $256 - 224 = 32$. The third octet, 33, falls in the block 32–63, so the subnet starts at 32.

Term	Value
Network address (classful, /16)	190.240.0.0
Subnetwork address (/19)	190.240.32.0

Question 6

A company is granted the class C address 201.70.64.0 and needs 6 subnets. Design the subnet mask and determine the borrowed bits, number of subnets, hosts per subnet, network addresses, broadcast addresses and valid host ranges.

Six is not a power of 2; the next power of 2 is $8 = 2^3$, so 3 bits must be borrowed from the 8 host bits of the default /24 mask. The new prefix is $24 + 3 = 27$, so the mask is 255.255.255.224, giving $2^3 = 8$ subnets (two spare, since only 6 were requested) and $32 - 27 = 5$ host bits per subnet, i.e. $2^5 = 32$ addresses per subnet, 30 of them usable once the network and broadcast addresses are removed. The block size is $256 - 224 = 32$.

Subnet	Network	Broadcast	Usable host range
1	201.70.64.0	201.70.64.31	201.70.64.1–201.70.64.30
2	201.70.64.32	201.70.64.63	201.70.64.33–201.70.64.62
3	201.70.64.64	201.70.64.95	201.70.64.65–201.70.64.94
4	201.70.64.96	201.70.64.127	201.70.64.97–201.70.64.126
5	201.70.64.128	201.70.64.159	201.70.64.129–201.70.64.158
6	201.70.64.160	201.70.64.191	201.70.64.161–201.70.64.190
7*	201.70.64.192	201.70.64.223	201.70.64.193–201.70.64.222
8*	201.70.64.224	201.70.64.255	201.70.64.225–201.70.64.254

* Subnets 7 and 8 are spare capacity, since only 6 subnets were requested.

Question 7

A company is granted the class B address 181.56.0.0 and needs 1000 subnets. Determine the borrowed bits, new subnet mask, prefix length, number of subnets and hosts per subnet.

$2^9 = 512$ is not enough, but $2^{10} = 1024$ is, so 10 bits are borrowed. The new prefix is $16 + 10 = 26$, giving the mask 255.255.255.192. There are $2^{10} = 1024$ subnets, and $32 - 26 = 6$ host bits per subnet, i.e. $2^6 = 64$ addresses per subnet, 62 usable.

Term	Value
Borrowed bits	10
New subnet mask	255.255.255.192 (/26)
Number of subnets	1024
Hosts per subnet	64 total, 62 usable

Question 8

An organisation is granted the block 130.56.0.0/16 and the administrator requires 1024 subnets. Find the number of addresses in each subnet, the subnet prefix, and the first and last address of the first and last subnets.

$2^{10} = 1024$ exactly, so 10 bits are borrowed and the new prefix is $16 + 10 = 26$, giving 255.255.255.192 and $2^6 = 64$ addresses per subnet. Borrowing splits the 16-bit host space (the third and fourth octets) into 1024 blocks of 64. The first block starts at offset 0, giving 130.56.0.0–130.56.0.63. The last block starts at offset $(1024 - 1) \times 64 = 65472$; dividing by 256 gives a third octet of 255 and a remainder of 192, so the last subnet is 130.56.255.192–130.56.255.255.

Term	Value
Addresses per subnet	64
Subnet prefix	/26 (255.255.255.192)
First subnet	130.56.0.0 to 130.56.0.63
Last subnet	130.56.255.192 to 130.56.255.255

Question 9

A small organisation is given the block 205.16.37.24/29. Find the first address, the last address and the number of addresses in the block, using both the binary method and the shortcut method.

Binary method: /29 is 255.255.255.248. ANDing the address with the mask gives the first address; ORing it with the inverse mask gives the last.

Line	Binary
Address	11001101 00010000 00100101 00011000
Mask	11111111 11111111 11111111 11111000
First (AND)	11001101 00010000 00100101 00011000
Last (OR inverse mask)	11001101 00010000 00100101 00011111

Shortcut method: the suffix length is 3, so the block size is $2^3 = 8$. The fourth octet, 24, is itself a multiple of 8, so it starts its own block, 24–31. Both methods agree: **first address** 205.16.37.24, **last address** 205.16.37.31, **8 addresses in total**.

Question 10

One address in a block is 167.199.170.82/27. Find the network address.

/27 is 255.255.255.224, so the block size is $256 - 224 = 32$. The fourth octet, 82, falls in the block 64–95, so the **network address is 167.199.170.64**.

Line	Binary			
Address	10100111	11000111	10101010	01010010
Mask	11111111	11111111	11111111	11100000
Result	10100111	11000111	10101010	01000000

Question 11

One address in a block is 205.16.37.39/28. Find the first address, last address, total addresses and usable host range, using the binary method and the mask method.

Binary method: /28 is 255.255.255.240. ANDing gives the first address, 205.16.37.32; ORing with the inverse mask (00000000 00000000 00000000 00001111) gives the last, 205.16.37.47.

Mask (shortcut) method: the block size is $256 - 240 = 16$; the fourth octet, 39, falls in the block 32–47, giving the same result. With $32 - 28 = 4$ host bits, the block holds $2^4 = 16$ addresses, 14 of them usable.

Term	Value
First address	205.16.37.32
Last address	205.16.37.47
Total addresses	16
Usable host range	205.16.37.33–205.16.37.46

Question 12

Each of the following addresses belongs to a block. Find the first address, last address and usable host range for each.

Address	First	Last	Usable host range
(a) 14.12.72.8/24	14.12.72.0	14.12.72.255	14.12.72.1–14.12.72.254
(b) 200.107.16.17/18	200.107.0.0	200.107.63.255	200.107.0.1–200.107.63.254
(c) 70.110.19.17/16	70.110.0.0	70.110.255.255	70.110.0.1–70.110.255.254

For (a), /24 fixes the first three octets and only the fourth resets. For (b), /18 is 255.255.192.0, giving a block size of $256 - 192 = 64$ in the third octet; 16 falls in the block 0–63. For (c), /16 fixes the first two octets and both remaining octets range over their full span.

Question 13

An organisation is granted the block 130.34.12.64/26 and requires 4 subnets. Design the subnets and show the address range of each.

/26 gives $32 - 26 = 6$ host bits, a block size of $2^6 = 64$, so the granted block spans 130.34.12.64–130.34.12.127. Four subnets need $\log_2 4 = 2$ more borrowed bits, giving the prefix /28 and a block size of $256 - 240 = 16$, which splits the 64-address range into four equal parts.

Subnet	Address	Range
1	130.34.12.64/28	130.34.12.64–130.34.12.79
2	130.34.12.80/28	130.34.12.80–130.34.12.95
3	130.34.12.96/28	130.34.12.96–130.34.12.111
4	130.34.12.112/28	130.34.12.112–130.34.12.127

Question 14

An organisation is granted the block 14.24.74.0/24 and needs three sub-blocks: one of 120 addresses, one of 60 and one of 10. Design the VLSM allocation.

VLSM always allocates the largest requirement first, to the smallest power-of-two block that is at least as big as the requirement, and continues from where the previous block ended.

Sub-block	Hosts needed	Prefix	Network address	Range
1	120	/25	14.24.74.0	14.24.74.0–14.24.74.127
2	60	/26	14.24.74.128	14.24.74.128–14.24.74.191
3	10	/28	14.24.74.192	14.24.74.192–14.24.74.207
Unused	—	—	14.24.74.208	14.24.74.208–14.24.74.255

The smallest power of 2 at least 120 is $128 = 2^7$ (/25); at least 60 is $64 = 2^6$ (/26); at least 10 is $16 = 2^4$ (/28). The three sub-blocks use $128 + 64 + 16 = 208$ addresses, leaving $256 - 208 = 48$ addresses unused.

Question 15

An organisation is granted the block 14.24.74.0/24 and needs three sub-blocks: one of 10 addresses, one of 60 and one of 120. Design the VLSM allocation.

The order in which the requirements are listed does not determine the allocation order: VLSM always sorts by size, largest first, because larger blocks need stricter address alignment. Sorting descending gives $120 \rightarrow 60 \rightarrow 10$, exactly as in Question 14, so the allocation is mathematically identical.

Requested size	Order	Prefix	Network address	Range
120	1st (largest)	/25	14.24.74.0	14.24.74.0–14.24.74.127
60	2nd	/26	14.24.74.128	14.24.74.128–14.24.74.191
10	3rd (smallest)	/28	14.24.74.192	14.24.74.192–14.24.74.207

Question 16

An ISP is granted the block 190.100.0.0/16 and must serve 64 customers requiring 256 addresses each, 128 customers requiring 128 addresses each, and 128 customers requiring 64 addresses each. Determine the prefix for each group, the address ranges allocated, the total addresses used and the addresses remaining. (This case is worked in full as the ISP example earlier in this section; the table below restates the result.)

Group	Customers	Addr./customer	Prefix	Range
A	64	256	/24	190.100.0.0–190.100.63.255 (16,384 addr.)
B	128	128	/25	190.100.64.0–190.100.127.255 (16,384 addr.)
C	128	64	/26	190.100.128.0–190.100.159.255 (8,192 addr.)

The block holds 65,536 addresses; 40,960 are allocated, leaving 24,576 addresses free in the range 190.100.160.0–190.100.255.255.

Question 17

An ISP is granted the block 16.12.64.0/20 and must allocate addresses to 8 organisations, each requiring 256 addresses. Find the number and range of addresses in the ISP block, the range allocated to each organisation, the unallocated range and a distribution outline.

With $32 - 20 = 12$ host bits, the block holds $2^{12} = 4096$ addresses. /20 is 255.255.240.0, giving a third-octet block size of $256 - 240 = 16$, so the ISP block spans 16.12.64.0–16.12.79.255. Since $256 = 2^8$, each organisation gets a /24.

Organisation	Range
Org 1	16.12.64.0/24 (16.12.64.0–16.12.64.255)
Org 2	16.12.65.0/24 (16.12.65.0–16.12.65.255)
Org 3	16.12.66.0/24 (16.12.66.0–16.12.66.255)
Org 4	16.12.67.0/24 (16.12.67.0–16.12.67.255)
Org 5	16.12.68.0/24 (16.12.68.0–16.12.68.255)
Org 6	16.12.69.0/24 (16.12.69.0–16.12.69.255)
Org 7	16.12.70.0/24 (16.12.70.0–16.12.70.255)
Org 8	16.12.71.0/24 (16.12.71.0–16.12.71.255)

The 8 organisations use $8 \times 256 = 2048$ addresses, leaving $4096 - 2048 = 2048$ unallocated in 16.12.72.0–16.12.79.255.

Question 18

An ISP is granted the block 80.70.56.0/21 and must allocate addresses to two organisations requiring 500 addresses each, two requiring 250 each and three requiring 50 each. Find the number and range of addresses in the ISP block, the allocation for each organisation, the unallocated range and a distribution outline.

With $32 - 21 = 11$ host bits, the block holds $2^{11} = 2048$ addresses. /21 is 255.255.248.0, so the third-octet block size is $256 - 248 = 8$; since 56 is already a multiple of 8, the ISP block spans 80.70.56.0–80.70.63.255. The smallest power-of-two prefix for 500 is $512 = 2^9$ (/23); for 250 it is $256 = 2^8$ (/24); for 50 it is $64 = 2^6$ (/26). Allocating largest first:

Organisation	Addresses needed	Prefix	Range
A1	500	/23	80.70.56.0–80.70.57.255
A2	500	/23	80.70.58.0–80.70.59.255
B1	250	/24	80.70.60.0–80.70.60.255
B2	250	/24	80.70.61.0–80.70.61.255
C1	50	/26	80.70.62.0–80.70.62.63
C2	50	/26	80.70.62.64–80.70.62.127
C3	50	/26	80.70.62.128–80.70.62.191

The allocation uses $(2 \times 512) + (2 \times 256) + (3 \times 64) = 1728$ addresses, leaving $2048 - 1728 = 320$ unallocated in 80.70.62.192–80.70.63.255.

Question 19

A supernet has first address 205.16.32.0 and mask 255.255.248.0. A router receives packets for 205.16.37.44, 205.16.42.56 and 205.17.33.76. Determine which packets belong to the supernet.

The mask has 21 ones, so the prefix is /21. The third-octet block size is $256 - 248 = 8$, giving the supernet range 205.16.32.0–205.16.39.255 (third octet 32–39).

Destination	Belongs to supernet?
205.16.37.44	Yes – third octet 37 lies in 32–39
205.16.42.56	No – third octet 42 is outside 32–39
205.17.33.76	No – second octet 17 does not match 16

Question 20

A supernet has first address 205.16.32.0 and mask 255.255.248.0. Find the prefix length, the number of class C blocks aggregated, the first address, the last address and the total number of addresses.

The mask has 21 ones, so the prefix is /21. Each class C block is a /24, so the difference $24 - 21 = 3$ gives $2^3 = 8$ blocks aggregated. The first address is given directly; the third-octet block size, $256 - 248 = 8$, gives a range of 32–39, so the last address is 205.16.39.255. With $32 - 21 = 11$ host bits, the total is $2^{11} = 2048$ addresses, matching 8×256 .

Term	Value
Prefix length	/21
Class C blocks aggregated	8
First address	205.16.32.0
Last address	205.16.39.255
Total addresses	2048

Question 21

A company needs approximately 600 addresses. For each of the following sets of class C blocks, determine whether a valid supernet can be formed and explain why. A set is valid only if the number of blocks is a power of 2, the blocks are contiguous, and the first block's network number is an exact multiple of the block count.

Set	Blocks	Power of 2?	Contiguous?	Result
(a) 198.47.32/33/34.0	3	No	Yes	Invalid – 3 is not a power of 2
(b) 198.47.32/42/52/62.0	4	Yes	No	Invalid – gaps of 10 between blocks
(c) 198.47.31/32/33/52.0	4	Yes	No	Invalid – block 52 is not contiguous with the rest
(d) 198.47.32/33/34/35.0	4	Yes	Yes	Valid

For (d), the starting third octet, 32, divides evenly by the block count 4, so the alignment rule also holds. The combined range is 198.47.32.0–198.47.35.255, giving $4 \times 256 = 1024$ addresses under the mask /22 (255.255.252.0) – enough to cover the company's requirement of about 600 addresses.

Question 22

Determine the supernet mask required to aggregate 16 contiguous class C networks. Show the binary calculation, the prefix length and the decimal mask.

Since $2^4 = 16$, four bits must be reclaimed from the network portion of the default /24 mask, giving a new prefix of $24 - 4 = 20$.

Line	Binary
Mask (/20)	11111111 11111111 11110000 00000000

Converting each octet gives the **supernet mask 255.255.240.0**. With $32 - 20 = 12$ host bits, the total is $2^{12} = 4096$ addresses, matching $16 \times 256 = 4096$.

Unit 3: Cloud Computing Fundamentals

3.1. Importance of Cloud Computing

Short description: Cloud computing lets an organisation obtain computing resources as a service instead of buying and running them itself.

Motivating example: Suppose tomorrow you want to start an online shopping website. What things do you need?

- Computer / server

- Storage
- Internet
- Database
- Operating system
- Software
- Security
- Application

The point: Every one of these can be rented from a cloud provider instead of bought and maintained, which is why cloud computing matters to a business getting started quickly.

3.2. Cloud Service Models

Short description: A cloud service model defines how much of the computing stack the provider manages on the consumer's behalf.

3.2.1 Overview

- **IaaS** – Infrastructure as a Service
- **PaaS** – Platform as a Service
- **SaaS** – Software as a Service

3.3. Infrastructure as a Service (IaaS)

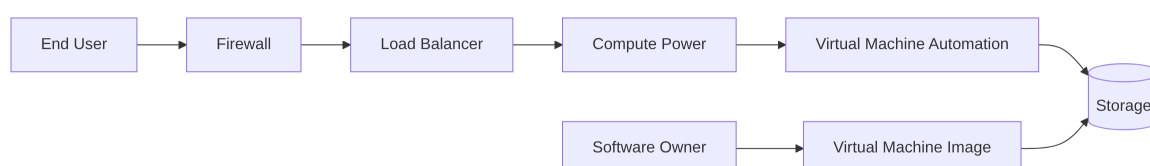
Short description: IaaS gives the provider's access to fundamental resources such as physical machines, virtual machines and virtual storage.

Also offered:

- Virtual machine disk storage
- Virtual local area networks (VLANs)
- Load balancers
- IP addresses
- Software bundles

How it works: All of the above resources are made available to the end user via server virtualization, and are accessed by the customer as if they own them.

3.3.1 Architecture diagram



3.3.2 Benefits

IaaS allows the cloud provider to freely locate the infrastructure over the Internet in a cost-effective manner. Key benefits:

- Full control of the computing resources through administrative access to VMs.
- Flexible and efficient renting of computer hardware.
- Portability and interoperability with legacy applications.

3.3.3 Issues

IaaS shares issues with PaaS and SaaS, such as network dependence and browser-based risks. It also has issues specific to itself:

Compatibility with legacy security vulnerabilities: Because IaaS lets the consumer run legacy software on the provider's infrastructure, it exposes the consumer to all of the security vulnerabilities of that legacy software.

Virtual machine sprawl: A VM can fall out of date with respect to security updates because IaaS allows the consumer to run, suspend or power off virtual machines at will. The provider can automatically update such VMs, but the mechanism is hard and complex.

Robustness of VM-level isolation: IaaS offers each consumer an isolated environment through the hypervisor, a software layer with hardware support for virtualization that splits one physical computer into multiple virtual machines.

Data erase practices: Consumers use virtual machines that share the provider's common disk resources. When a consumer releases a resource, the provider must ensure the next consumer to rent it does not see data residue from the previous one.

3.3.4 Characteristics

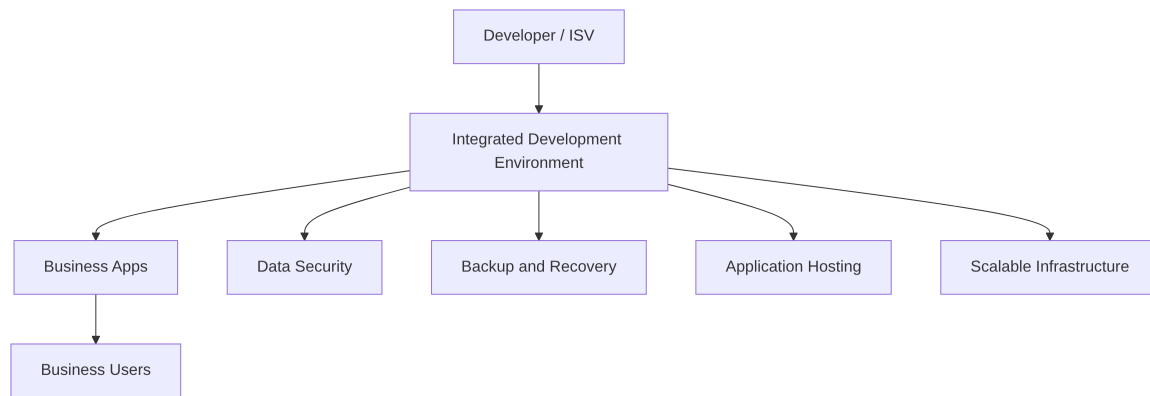
- Virtual machines with pre-installed software.
- Virtual machines with pre-installed operating systems, such as Windows or Linux.
- On-demand availability of resources.
- Ability to store copies of the same data in different locations.
- Computing resources that can be easily scaled up and down.

3.4. Platform as a Service (PaaS)

Short description: PaaS offers the development and deployment tools required to build applications, including point-and-click tools that let non-developers create web applications. Google App Engine and Force.com are example PaaS vendors – a developer logs into these platforms and uses their built-in API to build web applications.

Main drawback: Developer lock-in with a particular vendor. An application written in Python against Google's App Engine API is likely to work only in that environment, so vendor lock-in is the biggest problem in PaaS.

3.4.1 Architecture diagram



3.4.2 Benefits

Lower administrative overhead: The cloud provider is responsible for administration, so the consumer need not worry about it.

Lower total cost of ownership: The consumer need not purchase expensive hardware, servers, power or data storage.

Scalable solutions: Very easy to scale up or down automatically based on application resource demand.

More current system software: The provider is responsible for maintaining software versions and patch installations.

3.4.3 Issues

Lack of portability between PaaS clouds: Although standard languages are used, the implementation of platform services may vary. File, queue or hash-table interfaces of one platform may differ from another's, making it hard to move workloads between platforms.

Event based processor scheduling: PaaS applications are event oriented, which places resource constraints on them – they must answer a request within a given interval of time.

Security engineering of PaaS applications: Because PaaS applications depend on the network, they must explicitly use cryptography and manage security exposures.

3.4.4 Characteristics

- Browser-based development environment; developers create databases and edit application code via an API or point-and-click tools.
- Built-in security, scalability and web service interfaces.
- Built-in tools for defining workflow, approval processes and business rules.
- Easy to integrate with other applications on the same platform.
- Web service interfaces that connect applications outside the platform.

3.5. Software as a Service (SaaS)

Short description: SaaS provides a software application as a service to end users – software deployed on a hosted service and accessed via the Internet.

Example SaaS applications:

- Billing and invoicing systems
- Customer Relationship Management (CRM) applications
- Help desk applications
- Human Resource (HR) solutions

Customisation: Some SaaS applications are not customisable, such as an office suite. But SaaS also provides an API that lets a developer build a customised application on top.

3.5.1 Characteristics

- Software is made available over the Internet.
- The software is maintained by the vendor, not by where it runs.
- Licensing may be subscription based or usage based, and is billed on a recurring basis.
- Cost effective, since there is no maintenance burden on the end user.
- Available on demand, and can be scaled up or down on demand.
- Automatically upgraded and updated.
- Offers a shared data model, so multiple users share a single instance of the infrastructure without hard-coding functionality per user.
- All users run the same version of the software.

3.5.2 Benefits

- Modest software tools
- Efficient use of software licenses
- Centralised management and data
- Platform responsibilities managed by the provider
- Multitenant solutions

3.5.3 Issues

Browser based risks: If the consumer visits a malicious website and the browser is infected, subsequent access to a SaaS application can compromise the consumer's data. Using multiple browsers, dedicating one to SaaS access, or using a virtual desktop reduces the risk.

Network dependence: A SaaS application can only be delivered while the network is continuously available, and network reliability cannot be guaranteed by either the provider or the consumer.

Lack of portability between SaaS clouds: Transferring workloads from one SaaS cloud to another is not easy, because workflow, business logic, user interfaces and support scripts can all be provider specific.

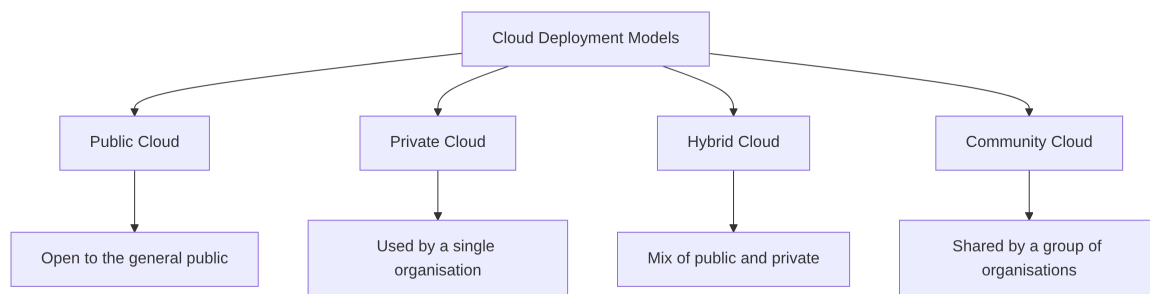
Service model examples

1. A company wants to host its website but does not want to purchase physical servers. It wants control over the operating system, storage and virtual machines. Identify the suitable cloud service model, justify the answer, and give one example.

2. A software development team wants to develop and deploy a web application without managing servers, operating systems or hardware. Identify the suitable cloud service model, explain why it is suitable, and give one example.
3. A college wants to provide students with online email and document-editing applications, accessed through a web browser without installing or maintaining software. Identify the suitable service model and give two reasons.

3.6. Cloud Deployment Models

3.6.1 Overview diagram



3.6.2 Public Cloud

Short description: The public cloud allows systems and services to be easily accessible to the general public – for example, Google, Amazon and Microsoft offer cloud services over the Internet.

Benefits:

- **Cost effective** – shares the same resources across a large number of consumers, so it has low cost.
- **Reliability** – draws on large numbers of resources from different locations, so a failed resource can be replaced by another one.
- **Flexibility** – easy to integrate with a private cloud, giving consumers a flexible approach.
- **Location independence** – services are delivered through the Internet.
- **Utility style costing** – pay-per-use, with resources accessible whenever the consumer needs them.
- **High scalability** – resources are made available on demand from a pool, and can be scaled up or down as required.

Disadvantages:

- **Low security** – data is hosted off-site on publicly shared resources, so it does not ensure a high level of security.
- **Less customisable** – comparatively less customisable than a private cloud.

3.6.3 Private Cloud

Short description: The private cloud allows systems and services to be accessible within a single organisation, which may manage it internally or through a third party.

Benefits:

- **Higher security and privacy** – not available to the general public, and resources are shared from a distinct pool, ensuring higher security and privacy.
- **More control** – more control over resources and hardware than a public cloud, because it is accessed only within the organisation.
- **Cost and energy efficiency** – not as cost effective as public clouds, but offers more efficiency than a public cloud.

Disadvantages:

- **Restricted area** – accessible only locally, and difficult to deploy globally.
- **Inflexible pricing** – purchasing new hardware to meet demand is very costly.
- **Limited scalability** – can be scaled only within the capacity of the internally hosted resources.

3.6.4 Hybrid Cloud

Short description: The hybrid cloud is a mixture of public and private cloud. Non-critical activities are performed using the public cloud, while critical activities are performed using the private cloud.

Benefits:

- **Scalability** – offers both public cloud scalability and private cloud scalability.
- **Flexibility** – offers both secure resources and scalable public resources.
- **Cost efficiencies** – public cloud is more cost effective than private, so the hybrid cloud gains this saving.
- **Security** – the private cloud portion ensures a higher degree of security.

Disadvantages:

- **Networking issues** – networking becomes complex due to the presence of both private and public cloud.
- **Security compliance** – it is necessary to ensure cloud services comply with the organisation's security policies.

3.6.5 Community Cloud

Short description: The community cloud allows systems and services to be accessible by a group of organisations, sharing infrastructure among several organisations from a specific community. It may be managed internally or by a third party.

Benefits:

- **Cost effective** – offers the same advantage as a private cloud at lower cost.
- **Sharing between organisations** – provides infrastructure to share cloud resources and capabilities among several organisations.
- **Security** – comparatively more secure than the public cloud.

Issues: Since all data is housed at one location, care is needed in storing data in a community cloud, because it might be accessible by others. Allocating responsibility for governance, security and cost is also challenging.

Deployment model examples

1. ABC E-Commerce has customers across India. During normal days it has 500 users, but during festival sales, users increase to 50,000. The company wants to quickly increase computing resources when demand increases and reduce them afterward, without purchasing and maintaining its own servers. Which cloud deployment model should ABC E-Commerce choose? Give a reason.
2. A government department stores highly confidential citizen information such as identity details, tax records and official documents. It requires complete control over its servers, network and security policies, and has its own IT team and budget to maintain infrastructure. Which cloud deployment model is most suitable? Why?
3. A bank wants to run its core banking system on infrastructure dedicated only to the bank, because of strict security and compliance requirements. However, it also wants to use cloud services for its public website and mobile application during periods of high traffic. Which cloud deployment model(s) should the bank use?
4. A company has two applications: Application A requires dedicated infrastructure and handles confidential employee data; Application B is an online shopping application whose users may increase from 1,000 to 100,000 during sales. Identify the suitable cloud deployment model for each application and justify the answer.

3.7. Regions, Availability Zones and Edge Locations

Short description: Cloud providers such as AWS, Microsoft Azure and Google Cloud have data centres distributed across the world. To organise this global infrastructure and provide reliable, fast and highly available services, they use the concepts of Regions, Availability Zones and Edge Locations.

3.7.1 Region

Definition: A Region is a geographical area of the world in which a cloud provider operates one or more data centres or Availability Zones. Providers divide their global infrastructure into regions so customers can choose where their cloud resources are deployed.

Example AWS regions: Mumbai, Singapore, Tokyo, London, Frankfurt, Northern Virginia, Sydney. Azure and Google Cloud have their own geographical regions.

Why regions are required: If a company operates in India with customers primarily located in India, but hosts its application in a distant location such as the United States, communication may take longer because data has to travel a greater distance. Deploying the application in an Indian region instead helps reduce network latency.

Characteristics:

- **Geographical location** – a region represents a specific geographical area.
- **Contains multiple Availability Zones** – a region is generally divided into multiple isolated Availability Zones.
- **Data residency** – organisations can choose a region based on where they want their data and applications hosted, subject to the provider's services and applicable regulations.
- **Disaster recovery** – organisations can use multiple regions for disaster recovery.
- **Reduced latency** – selecting a region closer to users reduces network latency.
- **Service availability** – not every cloud service is necessarily available in every region.

3.7.2 Availability Zone

Definition: An Availability Zone (AZ) is an isolated infrastructure location within a cloud Region. A Region contains multiple AZs, designed to operate independently enough that a failure in one AZ does not necessarily affect the others.

Why AZs are required: The main purpose of Availability Zones is high availability and fault tolerance. If a company runs its entire application in only one Availability Zone, and that AZ experiences a power failure, network failure, hardware failure, cooling failure or other infrastructure problem, the application may become unavailable. Distributing resources across multiple AZs avoids this.

Characteristics:

- **Isolation** – AZs are designed to be isolated from failures in other AZs.
- **Independent infrastructure** – each AZ has independent power, cooling, networking and physical facilities, reducing the chance a single failure affects all AZs.
- **High availability** – applications can be deployed across multiple AZs.
- **Fault tolerance** – using multiple AZs lets applications tolerate certain infrastructure failures.
- **Low-latency communication** – AZs within a region are connected by the provider's high-speed network, so applications distributed across AZs communicate efficiently.

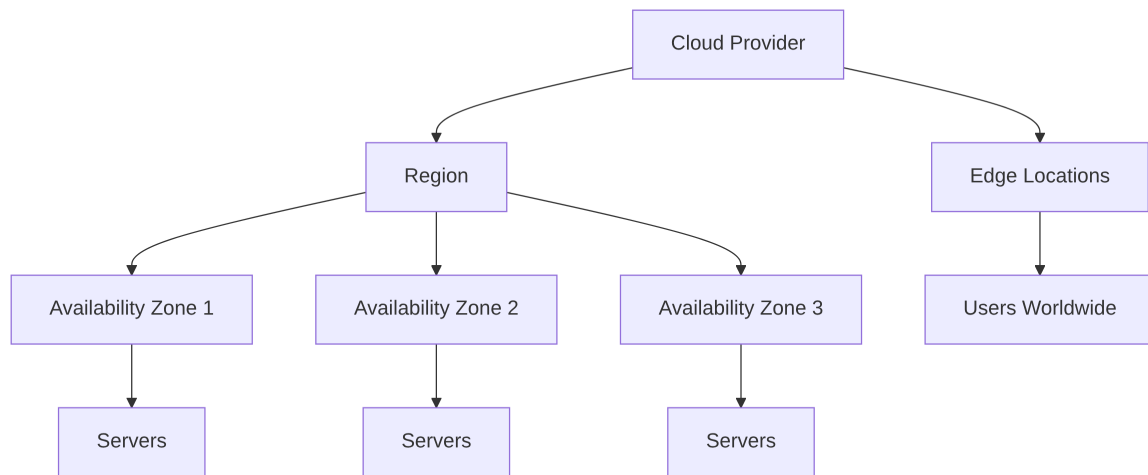
3.7.3 Edge Location

Definition: An Edge Location is a distributed location where a cloud provider's services can cache or deliver content closer to end users. Edge locations are particularly important for Content Delivery Networks (CDNs) – for example, AWS uses Amazon CloudFront to deliver content through a global network of edge locations.

Why edge locations are required: Consider a user in Nagpur accessing a website whose main server is in the United States. Without caching, the request must travel that long distance, resulting in higher latency. With an edge location, the user instead receives cached content from a location closer to them, giving faster delivery and lower latency.

How they work: A website's images, videos, CSS files, JavaScript files, web pages and other static content are cached across edge locations. When a user requests this content, the CDN serves it from the nearest appropriate edge location.

3.7.4 Relationship between Region, AZ and Edge Location



- A **Region** is where cloud resources are deployed.
- An **Availability Zone** provides isolation and redundancy within a Region.
- An **Edge Location** brings frequently accessed content and services closer to users.
- If one AZ fails, traffic is served by the other AZs, which ensures high availability and fault tolerance.

3.7.5 Difference between Region, AZ and Edge Location

Parameter	Region	Availability Zone	Edge Location
Meaning	Geographical area	Isolated infrastructure location within a Region	Location closer to users for content delivery / caching
Main purpose	Resource deployment	High availability	Reduce latency
Contains	Multiple AZs	Infrastructure / data centres	CDN / cache infrastructure
Geographical size	Large	Smaller	Distributed near users
Main benefit	Location and regional resilience	Fault tolerance	Faster content delivery
Example	Mumbai Region	AZ within Mumbai Region	CloudFront edge location
Mainly used for	Compute, storage, databases, etc.	Redundant application deployment	CDN / content delivery
Failure impact	Regional failure can affect resources in that region	Failure can affect resources in that AZ	Cached content may be served from another location, depending on the service or design

Practice problems

1. An AWS Region has 3 Availability Zones. Each AZ contains 20 servers. How many servers are available in the entire Region?
2. An e-commerce company uses 2 Regions. Each Region has 3 AZs, and each AZ has 50 servers. Find the total number of servers deployed.
3. A cloud provider has 3 Regions. Each Region contains 2 AZs. Each AZ has 25 servers. Additionally, the provider has 4 Edge Locations per Region. Find: (i) total AZs, (ii) total servers, (iii) total Edge Locations.

3.8. Cloud Computing: Benefits and Challenges

Benefits	Challenges
Cost saving – reduces hardware and maintenance costs.	Security and privacy – risk of data breaches and unauthorized access.
Scalability – resources can be increased or decreased as needed.	Internet dependency – requires a reliable Internet connection.
Accessibility – access data and applications from anywhere.	Downtime – cloud service failures can interrupt operations.
Flexibility – easily adapt resources according to requirements.	Vendor lock-in – difficult to switch between cloud providers.
High availability – provides reliable access using redundancy.	Limited control – less control over the underlying infrastructure.
Automatic updates – provider manages updates and maintenance.	Data transfer issues – large data transfers can be slow or costly.
Disaster recovery – enables backup and recovery of data.	Compliance – must meet legal and regulatory requirements.
Faster deployment – resources can be deployed quickly.	Performance variations – performance may vary due to network or shared resources.

3.8.1 Benefits, in detail

Reduced total cost of ownership:

- Cost-effective – reduces hardware, storage, bandwidth and software costs.
- Scalable – suitable for users and organisations of all sizes.
- Pay-as-you-use – pay only for the resources consumed.
- Easy access – resources are managed through web services and APIs.
- Additional services – supports websites, billing, shopping and shipping.
- Business enablement – helps small businesses operate like established enterprises.

Increased scalability and reliability:

- Leverages the cloud provider's global infrastructure.
- Provides backup and fault tolerance.
- Reduces latency and improves performance.
- Easily handles peak demand – especially beneficial for small and medium businesses.

Increased test automation:

- Enables continuous build and automated testing.
- Provides on-demand test servers at scale.
- Virtualization supports testing across different hardware and software configurations.
- Easily performs large-scale stress and performance testing.
- Pay-as-you-use makes extensive testing cost-effective.

Enabling collaborative applications:

- Enables quick and flexible business partnerships.
- Supports easy sharing of information among partners.
- Facilitates collaboration in emerging markets and niche opportunities.
- Cloud platforms enable access to shared CRM and business applications.
- Helps businesses collaborate without complex infrastructure.

Reduced middleware tax:

- Reduces the cost and complexity of middleware.
- Simplifies application development and deployment.
- Minimises dependency on complex frameworks and tools.
- Reduces the burden of legacy middleware infrastructure.

Increased application development agility:

- Reduces dependency on middleware, databases and IT operations.
- Enables developers to work end-to-end independently.
- Cloud services provide simple, easy-to-use APIs.
- Speeds up application development and deployment, and improves team flexibility.

Increased end-user computing:

- Enables users to create customised solutions for their needs.
- Uses simple technologies like JavaScript and REST APIs.
- Mashup tools help users build personalised applications.
- Makes legacy data easily accessible through standard protocols, reducing the need for special-purpose applications.

3.8.2 The Challenges

Service Level Agreements: Define the service quality, availability and performance expected, and clarify provider and customer responsibilities. They help manage business risks and reliability; SLA monitoring tools ensure compliance, which matters for enterprise cloud adoption.

National data protectionism: Governments are concerned about data stored outside national borders. Data centres may be located far from the data owner's country, and laws may restrict data storage and transfer across borders, creating challenges for global cloud adoption and compliance.

Application security: Application developers are responsible for security beyond authorization. Critical security should be handled outside the browser, using strong encryption for data in transit and at rest and following SSL and other security best practices. Cloud platforms can provide enterprise-level security.

Loosely coupled service design: Enables applications to work with minimal dependency on each other, requiring standard protocols and consistent service design. ATOM publishing provides a simple service model, though developers must still manage different protocols and service interactions.

Query oriented versus API oriented programming: Cloud computing increasingly uses query-oriented programming. Technologies like MapReduce, LINQ and stream processing process large datasets using SQL/XQuery-style operations instead of complex APIs, so developers need skills in complex queries and data processing.

Occasionally disconnected operation:

- **Offline capability** – mobile applications must work offline and auto-sync with the cloud upon reconnection.
- **Client-side execution** – Google Gears runs web models locally on the client with server sync.
- **Sync frameworks** – Microsoft Azure offers flexible frameworks supporting varied data and file sync patterns.